



COSC2767|COSC2805 | System Deployment and Operations

Assignment 2 - Team BugOps

Students:

Tran Phan Hoang Phuc – s3929597
Nguyen Doan Trung Truc – s3974820
Phan Manh Ha -
Ho Tuan -
Le Duc Huy -

Tutorial Session: Thursday 2:30 PM

Lecturers: Dr. Tom Huynh

Due Date: Sep 19th – 17:00

"We declare that in submitting all work for this assessment we have read, understood and agree to the content and expectations of the [Assessment declaration \(Links to an external site.\)](#)"

Table of Contents:

I. The CI/CD Pipeline Solution	4
II. Main Requirement	9
2.1) Cloud Integration:	9
2.1.1) Solution Overview:	9
2.1.2) Configuration:	9
2.2) Containerized Microservices:	10
2.2.1) Solution Overview:	10
2.2.2) Configuration:	10
2.3) Continuous Integration:	11
2.3.1) Solution Overview:	11
2.3.2) Configuration and Settings:	11
2.4) Continuous Delivery:	12
2.4.1) Solution Overview:	12
2.5) Testing Framework:	16
2.5.1) Solution Architecture:	16
2.5.2) Configuration and Settings:	16
2.6) Automation Tool:	17
2.6.1) Solution Overview	17
2.6.2) Configurations andSettings	18
2.7) Configuration Management:	18
2.7.1) Solution Overview:	18
2.7.2) Configuration:	19
2.8) Automated Alerting System:	19
2.9) Orchestration	20
III. Advanced Requirement	20
3.1) Infrastructure as Code (IaC):	20
3.1.1) Solution Overview:	21
3.1.2) Configurations & Settings:	21
3.2) Monitoring and Alerting:	21
3.2.1) Solution Overview:	21
3.2.2) Configurations & Settings:	22
3.4) Advanced Deployment Strategy:	22
3.4.1) Solution Overview:	22
3.4.2) Configurations & Settings:	22
3.5) Scalable Infrastructure with Auto-Scaling	23

IV. Application Flow	23
4.1) Technical Workflow Architecture	23
4.1.1) Automated Activation Through Intelligent Integration	23
4.1.2) Complete Process Flow: Development to Production	23
4.1.3) Blue-Green Deployment Strategy Implementation	24
4.1.4) Real-Time Monitoring Infrastructure	24
4.2) Automation Achievement: Single-Click Customer Access	25
V. Known Bugs/Problems	25
VI. Self-evaluation and Reflection of your CI/CD pipeline	27
6.1) Self-Evaluation	27
6.2) Industry Practices Reflection	28
6.3) Effectiveness in Handling Failures & Scalability	28
6.4) Achieving Key Aspects	28
6.5) Why This Solution is the Best Fit	29
VII. Conclusion	29
VIII. Project Responsibilities	30
IX. References	32
X. Appendices	34

I. The CI/CD Pipeline Solution

Tom is a skilled web developer building an **RMIT eCommerce Store** on the MERN platform, but he lacks a production-level DevOps workflow. The resulting issues that cause the project to stall and crash unexpectedly include *manual deployments, inconsistent testing, limited observability, and a lack of reliable rollback mechanisms that create risk, slow releases, and scalability bottlenecks*. To address most of these pain points, this project aims to build an automated CI/CD pipeline on AWS so Tom can focus on coding - with a simple workflow of *each push triggering automated builds, tests, container images, and staging/testing deployments to production, with monitoring/alerting and secure rollbacks*. The result is a repeatable, reliable, and scalable release process for his application's frontend, backend, and database.

To provide a suitable and budget-friendly solution for Tom, we designed the pipeline based on the project's needs [1]. In the current scope, our goal is to deliver a **basic and efficient solution** that fits within the sustainable budget, so we propose a “**standard pipeline**” that includes:

- + **Jenkins:** This will be the backbone of our **CI/CD pipeline**, enabling both **Continuous Integration (CI)** and **Continuous Delivery (CD)**. A key feature is “**Pipeline as Code**”, which allows programmatic handling, simplifying setup management and tracking by storing pipeline configurations in the source code [2]. Additionally, Jenkins is open-source and supports **over 2000 community-contributed plugins**, giving Tom and the DevOps team the flexibility to add tools and enhance future projects [3]. It is also the **master-slave architecture**, where the master node manages the overall pipeline, and slave nodes perform the actual work (building, testing, deploying), which helps distribute workloads into smaller stages for easier debugging and extension, allowing future additions like testing or deployment stages [2], [3]. While the single-server setup can sometimes lead to **memory overload** [4], the **EC2 instance setup** helps share the workload, reducing the potential for such issues and providing an efficient solution for the current process [4], [5]. Furthermore, with **Blue Ocean pipeline visualization capabilities**, Jenkins can help track processes more clearly and easily know and detect parts that have or need improvement. This setup is useful for Tom's project, allowing multiple stages to be handled concurrently.
- + **Docker and Dockerhub:** It is a powerful partner of Jenkins, by handling **OS compatibility issues** [6]. It ensures that the development and deployment environments are consistent, reducing configuration issues. Developers simply write and push code to **Jenkins**, which then creates Docker images for deployment, ensuring smooth containerization across environments. This **containerized automation** enhances the efficiency of building, testing, and deploying applications, providing a reliable and robust solution [7].

- + **Kubernetes:** To handle load balancing, distribution across multiple servers, and scaling the cluster up or down, Kubernetes was chosen [8]. Kubernetes ensures that each service (frontend, backend, and database) is hosted in its own pod within the cluster, with replicas scaling based on demand. Kubernetes's load-balancing feature ensures that incoming traffic is evenly distributed across replicas, helping to manage high traffic and ensuring that users can always access the application smoothly. Also use of **Horizontal Pod Autoscaling** in Kubernetes automatically adjusts the number of pods based on CPU/memory utilization or custom metrics. [8], [9], which directly addresses Tom's scalability bottlenecks. Although Kubernetes can be more complex to set up than Docker Swarm [9], its superior scalability and powerful load-balancing features make it the ideal solution for Tom's needs. With all of the above in place, Tom can manage the high traffic demands of his e-commerce store without worrying about downtime with this tool.
- + **Playwright:** As a popular **web UI and end-to-end (e2e) testing tool**, Playwright offers efficient **parallel execution and automation** [10]. It allows Tom's tests to run *concurrently, reducing execution time*, especially for time-consuming e2e tests. By automating tests across multiple browsers, Playwright helps **quickly identify UI issues**, streamlining debugging and improving testing efficiency. Developers design tests, and Playwright automatically runs them, ensuring high efficiency. It also **captures photos and videos** during testing, providing insights into **user behavior and app performance**. This will help Tom easily *manage UI and e2e bugs*. Tests are written in simple scripts that automate browser actions like clicking, typing, and navigating, simulating real user interactions.
- + **Jest + Supertest:** For **unit and integration testing**, **Jest** and **Supertest** are used. **Jest** is a powerful testing framework that provides **mocking, assertions, and snapshot testing**. It allows for efficient **unit testing** by mocking database interactions and ensuring that the backend logic functions correctly. **Supertest** is used to test the **integration** of the backend API with the **database** by simulating HTTP requests, allowing us to verify that the API routes handle requests and responses as expected.
 - **Unit Testing with Jest:** Mock database functions to test the business logic.
 - **Integration Testing with Supertest:** Simulate real HTTP requests to test API routes and verify their interaction with the database.

These tools offer a scalable and modular testing framework, ensuring that the code is reliable and can handle real-world usage scenarios effectively [11].

- + **Jenkin's Email Extension and Slack Notification plugins:** To help Tom and the development team stay informed about the deployment process, these tools have been integrated into the CI/CD pipeline [12]. These alerting systems **notify the team**

immediately upon the occurrence of any issues during the deployment process, allowing for **fast response times and faster troubleshooting [12]**. As Jenkins plugins, these tools ensure that the pipeline can *seamlessly stop when a problem arises*, triggering automatic notifications via email or Slack. This integration *improves team collaboration* and ensures that the development team can *resolve issues more efficiently*, minimizing downtime and maintaining smooth operations.

- + **AWS EC2-Instance:** In this project, they will be used **as remote virtual machines** to host and integrate tools in the CI/CD pipeline. Because it provides a **cost-effective and scalable solution**, reduces dependency on local infrastructure, and improves the **reliability and scalability** of the project [13], it will help the DevOps team to **quickly scale** based on demand to **optimize costs and performance**. Additionally, EC2 is supported by **Elastic Load Balancing** to distribute traffic and **Auto Scaling with EC2 Auto Scaling Groups** to handle increased load, ensuring seamless performance even during peak periods [13]. By using EC2, the team can manage the project in an elastic cloud environment with *minimal complexity*.
- + **Prometheus and Grafana:** These are powerful tools for **monitoring and visualizing system performance**. Prometheus, an open-source monitoring system that **stores and collects time-series data** from services, helps to **record important metrics** such as CPU utilization, memory utilization, request latency, and error rates [14]. This data is collected at regular intervals to **gain real-time insights** into the health of the system, which the team can analyze and decide on further scalability or load balancing. Secondly, Prometheus alerts the team when *safety thresholds are breached*, allowing for proactive **incident detection and minimizing downtime**. Furthermore, with Grafana, it complements Prometheus by allowing for **advanced visualization** of the collected metrics [14]. Together, they enable the team to **create interactive dashboards** that display *health and performance trends* for various components of the system, including backend services, databases, and user interfaces [14]. In Tom's project, this integration ensures that any *bottlenecks or performance degradations* can be **easily identified and resolved**. By presenting system metrics in a clear, granular manner, Prometheus and Grafana enable Tom and the DevOps team to maintain a **stable, scalable, high-performance process**, with faster debugging and improved overall system stability [14].
- + **Blue-Green Deployment Strategy:** In addition to the current CI/CD pipeline tools, a **blue-green deployment** strategy could further enhance **reliable updates** and reduce downtime. This strategy involves having two identical production environments (blue and green). The **blue** environment serves live traffic while the **green** environment is used for staging the new version of the application [15], [16]. Once the new version is tested and validated in the green environment, traffic is switched to green, minimizing

disruptions. This strategy can be incorporated into the pipeline using tools like **Kubernetes** for traffic management, ensuring a smoother and **safer deployment process** [15], [16].

- + **Helm:** As a good partner to Kubernetes in **package management**, Helm simplifies the management and deployment of applications on Kubernetes clusters [17], [18]. Helm uses ***YAML files and templates*** to generate configuration files based on parameters, which is a huge benefit for both DevOps teams and developers by **increasing productivity by automatically generating configuration files, ensuring everything is consistent, and minimizing the possibility of errors** [17]. Furthermore, Helm is also great for scalability, allowing the team to easily update information like replica counts without having to make manual changes [18]. Additionally, Helm has a **previous stable version rollback feature**, making it easy to roll back to a previous stable version if something goes wrong, keeping everything running smoothly. With Helm charts, teams can **standardize deployment and scale applications easily** while maintaining flexibility. With those advantages, Tom will be able to help a lot in ensuring his project runs smoothly.
- + **Ansible:** In this project, Ansible is used primarily for **managing server and cloud instance configurations**, which greatly ***reduces the manual tasks*** that developers often encounter [19], [20]. With Ansible, the DevOps team was able to **automate infrastructure setup through YAML files**, making it easier to configure and manage environments [19]. This tool ensures configurations are **standardized, repeatable, and easily modified** without external access [19], [20]. Ansible also helps streamline processes, allowing development teams to scale or adjust environments as needed. Therefore, in this project, Ansible served as the first step towards infrastructure automation, allowing Tom and the web development team to minimize manual work by simply executing predefined commands to automatically set up the required infrastructure components.
- + **AWS CLI:** Since the project is built on **AWS Classroom**, the **DevOps team** needed a tool to manage **access keys** and **key IDs** and automate the creation of **EC2 instances** directly from the source code. This helps to reduce manual work and makes the pipeline more **automated** and **seamless**. **AWS CLI** has been installed to ensure that all **cloud services** can be provisioned through source code, streamlining the infrastructure setup and management [21].

With these tools and technologies, the DevOps team has designed some **main features of this pipeline**, including:

Feature	Tools/Technologies	How It Works
---------	--------------------	--------------

CI/ CD Automation	Jenkins	Automates the process of building, testing, and deploying code. Each push triggers the pipeline for continuous integration and delivery.
Containerization and Deployment	Docker and Kubernetes	Docker packages the app into containers, ensuring consistency across environments. Kubernetes manages scaling, load balancing, and distribution across multiple servers.
Infrastructure as Code	AWS CLI, Ansible	AWS CLI makes it easier to create and manage AWS resources, like EC2 instances , through source code. Ansible automates the configuration management of servers, services, and infrastructure through YAML files.
Package Management and Deployment	Helm	Helm manages Kubernetes applications through charts, automating the installation and management of Kubernetes resources in a predictable and reproducible way.
Scalable Infrastructure	EC2 (AWS) and Kubernetes	EC2 instances provide scalable cloud infrastructure, allowing the DevOps team to quickly scale based on demand while optimizing costs and performance.
Testing	Playwright, Jest, and Supertest	Playwright automates end-to-end tests for frontend and UI. Jest handles unit testing, while Supertest simulates HTTP requests for integration tests between the backend and database.
Monitoring and Alerts	Prometheus and Grafana	Prometheus collects system metrics like CPU, memory, and latency. Grafana visualizes these metrics on interactive dashboards , helping to identify and resolve issues quickly.
Version Control and Notifications	GitHub, Jenkins, Email and Slack Plugins	GitHub stores the code and triggers Jenkins . Email and Slack plugins send notifications about the pipeline status, ensuring fast responses to failures.

As this project is built within the **budget provided for students**, the DevOps team has aimed to **maximize the use of free tools** to keep costs low, utilizing tools like ***Jenkins, Docker, Playwright, and Prometheus*** while avoiding costly solutions like EKS or Atlas. While this setup is theoretically scalable, it is designed to handle **5 microservices** for now. This ensures a **scalable, efficient solution** that fits within the project's constraints and is appropriate for the current scope.

II. Main Requirement

2.1) Cloud Integration:

To maximize the benefits of **AWS services** for **performance** and **scalability**, **AWS EC2 instances** are used as virtual machines (VMs) to host and run **Jenkins** and all other necessary tools for **application deployment** and **CI/CD pipeline configuration**. The creation and configuration of these instances are automated using **AWS CLI**, ensuring minimal **manual work**, efficient **automation**, and reducing delays caused by manual setup. This also allows the **DevOps team** to scale the infrastructure easily without worrying about **performance variations** or unnecessary complexity. Moreover, **AWS EC2 Auto Scaling** is also helping Tom deal with scalability problems. To further enhance the deployment environment, **K3s** is used as a lightweight **Kubernetes** distribution for **container orchestration**, ensuring a fast and efficient setup for smaller clusters, reducing overhead while maintaining scalability. **K3s** integrates well with **AWS EC2** to provide container orchestration while keeping resource usage minimal.

2.1.1) Solution Overview:

- **Tools Used:** AWS EC2, AWS CLI, K3s Medium
- In this project, all work is done on EC2 instances to ensure a high-performance, scalable environment. By using EC2, we take full advantage of AWS's scalability, allowing the team to adjust resources as needed and optimize for both cost and performance.

2.1.2) Configuration:

- Each instance is configured and controlled using **AWS CLI** in the source code. The instances are provisioned as **t3.small** and **t3.medium** based on task division with the **Amazon Linux 2023 AMI (ami-amazon-linux-latest/al2023-ami-kernel-6.1-x86_64)**, as specified in the *provision-aws.yml* file, shown in **Figure 1** in the **APPENDIX**.
- **Security Groups:** Each **EC2 instance** is assigned a unique **security group** as defined in *provision-aws.yml*, shown in **Figure 2** in the **APPENDIX**. This ensures secure

communication between instances, allowing only the **necessary ports** to be open and mitigating security risks.

2.2) Containerized Microservices:

To address Tom's **Inefficient Collaboration** and **Deployment Risk problems**, the DevOps team has containerized each component of his project (front-end, back-end, and database) by using **Docker** and orchestrated these containers with **Kubernetes**. This ensures that each service runs in a **consistent environment** across all stages (development, staging, and production), eliminating the "*it works on my machine*" problem [7], [8]. Moreover, **Kubernetes Horizontal Pod Autoscaling** automatically adjusts the number of pods for each service, ensuring that the platform can scale according to demand [8].

2.2.1) Solution Overview:

- **Tools Used:** Docker, Docker Hub, Kubernetes.
- **Docker Images:** Each component is packaged as a **Docker image** (a read-only package containing code, runtime, libraries, and configuration) and stored in **DockerHub** [6], [7] **showing in Figure 3 in APPENDIX**. When run, it becomes a **container** (a live instance of the image). This guarantees that every service runs in the same environment, regardless of where it is deployed.
- **Kubernetes Orchestration:** Kubernetes manages the lifecycle, scaling, and networking of these Docker containers. It automatically handles scaling based on traffic and ensures that the services are always running, minimizing downtime and deployment risks [9].

2.2.2) Configuration:

- **Dockerfile:** Each service (front-end and back-end) has a **Dockerfile** that defines the base image, copies the code, installs dependencies, and specifies the startup command. **Figure 4** in the **APPENDIX** shows the Dockerfile setup.
- **Kubernetes Manifests:** Helm is the devops team's choice for managing continuous deployments to Kubernetes, ensuring a scalable and repeatable process. Helm will deploy a Helm chart with files like *backend-deploy-blue.yaml* and *frontend-deploy-green.yaml* **as shown in Figures 5, 6, and 7 in APPENDIX**, specifying the Docker container images and replica counts for each of our frontend and backend services. Additionally, a *service.yaml* file will be created to expose each service to traffic using a LoadBalancer to evenly distribute traffic. **Yaml values files** will be the common specification for any configuration in our project. Two configurations can be specified: *values-prod.yaml* (which specifies production settings like replica counts, image tags, etc.) and another

configuration *values-staging.yaml* (for staging environments), which can be deployed using the line `helm upgrade --install rmit-store ./helm/rmit-store -f values-.yaml`. This creates an easy, scalable deployment process that also allows for rollback to a Healthy state. **Figure 8 and Figure 9** in the **APPENDIX** provide a visual representation of the Helm and Kubernetes pipeline.

This containerized approach ensures that Tom's project is **consistently deployed, scalable, and more reliable, addressing both collaboration inefficiencies and deployment risks.**

2.3) Continuous Integration:

In order to reduce **Tom's manual deployment process, along with stress and burnout**, the DevOps team decided to integrate **Jenkins and a GitHub webhook**. This will **automate the development workflow, streamline the process**, lessen the manual effort, and **improve the reliability and stability** of the code base.

2.3.1) Solution Overview:

- **Tools Used:** Jenkins, GitHub, Jenkinsfile (Pipeline as Code).
- A **Jenkins-based Continuous Integration (CI)** process is set up with **GitHub webhooks**. Each time a developer *pushes code to the GitHub repository*, the webhook tells Jenkins to **initiate the pipeline**. Therefore, Tom's team will only need to push code to GitHub. Jenkins will **automatically pull the latest code**, build it, and run the steps necessary to ensure their code is both **built and tested with unit tests that the DevOps team has already set in source code**. The pipeline continues with deployment to staging and production, resulting in a completely automated process. This automated workflow ensures that developers like Tom can **focus on writing code** without worrying about **managing the deployment process**. The CI process streamlines the workflow, ensuring that the codebase remains stable and dependable.

2.3.2) Configuration and Settings:

- **Jenkinsfile:** The entire CI process is defined in the **Jenkinsfile**, which is stored in the root of the GitHub repository. This file outlines each step of the pipeline and makes the process **versioned, reusable, and manageable**. See **Figure 10 in the APPENDIX - Jenkinsfile configuration**.
- **GitHub Webhook:** A **webhook** is configured in the GitHub repository settings. It is set to trigger Jenkins whenever a push event occurs, notifying Jenkins to start the pipeline. By doing it this way, the pipeline can run automatically every time code is pushed. **Figure 11 in the APPENDIX** shows the webhook configuration.

This CI process helps Tom and his team by automating builds, tests, and deployments, *reducing manual effort, stress, and the risk of errors.*

2.4) Continuous Delivery:

Continuous Delivery (CD) is the *automated process of releasing code to users*. After the code is built and pulled in CI, the CD pipeline takes over to make the release process *fast, safe, and reliable*. It uses a series of steps to ensure **only good code** reaches production without **causing any downtime**.

2.4.1) Solution Overview:

- **Tools Used:** Jenkins, Docker, Helm, Kubernetes.

1. Prepare the App Package:

- The first thing Jenkins does is take the working code and package it in a Docker image, which is a way of saying it is put into a sealed box (image), with all the necessary components included to run the application.
- This image also gets assigned a version number and is sent to our online repo for Docker images, which is called Docker Hub.

```
stage('Docker GC'){
  steps{
    sh '''
      echo "[docker-gc] Before:" && docker system df || true
      # Keep image/build cache for faster builds
      docker container prune -f || true
      docker volume prune -f || true
      echo "[docker-gc] After:" && docker system df || true
    '''
  }
}

stage('Build images'){
  steps {
    sh '''
      docker build --pull $([ "${CLEAN_BUILD}" = "true" ] && echo "--no-cache") -t ${BACK_IMG}:${GIT_COMMIT} ./server
      # Build frontend with API_URL pointing to same-origin /api
      docker build --pull $([ "${CLEAN_BUILD}" = "true" ] && echo "--no-cache") --build-arg API_URL=/api -t ${FRONT_IMG}:${GIT_COMMIT} ./client
    '''
  }
}

stage('Push to Docker Hub'){
  steps {
    withCredentials([usernamePassword(credentialsId: 'dockerhub', usernameVariable: 'DH_USER', passwordVariable: 'DH_TOKEN')])
    sh '''
      echo "$DH_TOKEN" | docker login -u "$DH_USER" --password-stdin
      docker push ${BACK_IMG}:${GIT_COMMIT}
      docker push ${FRONT_IMG}:${GIT_COMMIT}
    '''
  }
}
```

Figure 1: Jenkins CI/CD Pipeline - Docker Build and Push Stages

2. Deploy to Staging:

- Next, the pipeline automatically deploys the Docker image to a **staging server**. This is a private test environment that is an exact copy of the live production server.
- We use a tool called **Helm** to simplify and standardize this deployment

```
stage('Deploy STAGING'){
  steps {
    withCredentials([file(credentialsId: 'kubeconfig-master', variable: 'KUBECONF')]){
      sh '''
        set -e
        cp "$KUBECONF" kubeconfig && chmod 600 kubeconfig

        # kubectl create ns if missing
        docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubectl:v1.30.6 get ns staging || \
        docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubectl:v1.30.6 create ns staging

        # Determine active color (green on first install so Service has endpoints)
        ACTIVE_COLOR=blue
        if ! docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubectl:v1.30.6 -n staging get svc backend >/dev/null 2>&1; then
          ACTIVE_COLOR=green
        fi

        # helm upgrade/install (mount repo root)
        MONGO_FLAG=""; [ -n "${MONGO_URI}" ] && MONGO_FLAG="--set-string mongo.uri=${MONGO_URI}";
        if [ -z "$STAGING_HOST" ]; then
          echo "STAGING_HOST is not set. Provide via JCasC env or .env → Makefile inventory."
          exit 1
        fi
      '''
    }
  }
}
```

Figure 2: Jenkins Pipeline - Staging Deployment Configuration

3. Test in Staging:

- Once the app is running in staging, we use **Playwright** to run automated **end-to-end and webUI tests** because the DevOps team wants to accurately deliver the performance of these tests in a real-time scenario - these tests ensure that the web can provide the most user-friendly experience.
- These tests act like real users—clicking buttons, filling out forms, and buying products—to make sure everything works perfectly.
- If any test fails, the pipeline stops immediately and alerts the team via **email**. This prevents bad code from moving forward.

```

stage('Smoke STAGING'){
  steps {
    sh '''
      if [ -z "$STAGING_HOST" ]; then
        echo "STAGING_HOST is not set. Provide via JCasC env or .env → Makefile inventory."
        exit 1
      fi
      STAGING_URL="http://$STAGING_HOST/api/health"
      echo "Staging smoke URL: $STAGING_URL"
      docker run --rm curlimages/curl:8.8.0 -fsS "$STAGING_URL"
    '''
  }
}

stage('E2E STAGING (Playwright)'){
  steps {
    catchError(buildResult: 'UNSTABLE', stageResult: 'UNSTABLE') {
      sh '''
        set -e
        # Run Playwright tests against staging in a container
        docker run --rm \
          -e BASE_URL="http://$STAGING_HOST" \
          -e ADMIN_EMAIL="${ADMIN_EMAIL:-admin@rmit.local}" \
          -e ADMIN_PASS="${ADMIN_PASS:-Admin123!}" \
          -e READ_ONLY_GUARD=0 \
          -v "$PWD"/tests/e2e-playwright:/e2e -w /e2e \
'''
    }
  }
}

```

Figure 3: Jenkins Pipeline - Automated Testing Stages

4. Deploy to Production (Zero Downtime):

- If all tests pass, the code is ready for users. The DevOps team uses a smart method called **Blue-Green Deployment**, which will be explained in the Advanced Requirement Part, to update the live site without anyone noticing.

```

e('Cutover BLUE -> GREEN'){
  en { expression { return (env.E2E_OK ?: '') == '1' } }
  eps {
    withCredentials([file(credentialsId: 'kubeconfig-master', variable: 'KUBECONF')]){
      sh '''
        set -e
        cp "$KUBECONF" kubeconfig && chmod 600 kubeconfig
        docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubectl:v1.30.6 -n prod patch service backend -p '{"spec":{"selector":{"app":"backend","activeColor":"green"}}}'
        docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubectl:v1.30.6 -n prod patch service frontend -p '{"spec":{"selector":{"app":"frontend","activeColor":"green"}}}'
        shred -u kubeconfig || rm -f kubeconfig
      '''
    }
  }
}

```

Figure 4: Jenkins Pipeline - Blue-Green Production Deployment

5. Safety Net and Rollback:

- The old Blue server is kept running for a short time as a backup.

- If we notice any problems with the new version, we can immediately switch traffic back to the stable Blue server. This gives us a quick and safe way to undo the release if needed.

```
stage('Verify ROLLBACK (staging)'){
  when { expression { return (env.ROLLED_BACK_STAGING ?: '') == '1' } }
  steps {
    withCredentials([file(credentialsId: 'kubeconfig-master', variable: 'KUBECONF')]){
      sh '''
        set -e
        cp "$KUBECONF" kubeconfig && chmod 600 kubeconfig
        echo "[verify] staging backend selector:"
        BK=$(docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubectl:v1.30.6 -n staging get svc backend -o jsonpath='{.spec.selector.activeColor}')
        echo "$BK" && [ "$BK" = "blue" ]
        echo "[verify] staging frontend selector:"
        FE=$(docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubectl:v1.30.6 -n staging get svc frontend -o jsonpath='{.spec.selector.activeColor}')
        echo "$FE" && [ "$FE" = "blue" ]
        shred -u kubeconfig || rm -f kubeconfig
        : ${STAGING_HOST:?STAGING_HOST not set}
        docker run --rm curlimages/curl:8.8.0 -fsS "http://$STAGING_HOST/api/health" > /dev/null
      '''
    }
  }
}
```

Figure 5: Jenkins Pipeline - Production Rollback Verification

Configurations & Settings:

- **Helm Charts:** We use Helm charts to package our application for Kubernetes that is already shown in **Figure 5,6 and 7 in the APPENDIX**. This makes deployments repeatable and easy to manage for both staging and production.
- **Kubernetes Labels:** Simple labels are used to tell Kubernetes where to send user traffic. We just updated this label to switch from the Blue server to the Green one.
- **jenkinsfile:** The entire CD process is written as code in the Jenkinsfile. It acts as the master plan that tells Jenkins exactly what to do at each step

With the support of these tools, Tom and the web development team will have a good experience when deploying the application - the entire CI/CD project is also **visualized on Jenkins with Blue Ocean** helping the team to easily monitor and identify issues earlier.

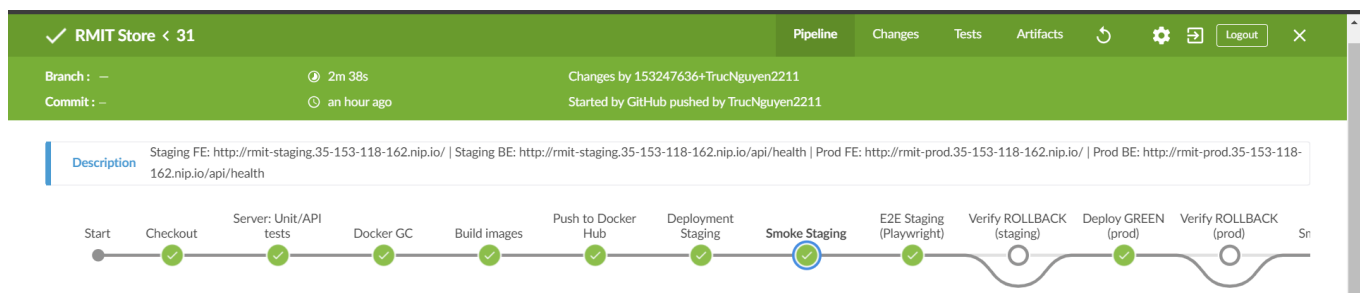


Figure 6: Jenkins Blue Ocean - Complete CI/CD Pipeline Visualization

2.5) Testing Framework:

Testing Framework Overview: The DevOps team implemented a comprehensive automated testing strategy using Jest, Supertest, and Playwright to ensure complete application validation. This multi-layered approach provides thorough coverage across unit, integration, and end-to-end testing scenarios, guaranteeing application reliability and functionality before deployment. Thus, providing Tom with an easy and efficient approach to testing the pipeline and website.

2.5.1) Solution Architecture:

- **Tools Used:** Jest, Supertest, Playwright.
- **Jest and SuperTest:** These are used in backend API and database testing. Jest is used in executing unit tests for the backend code to confirm that all the API endpoints behave as they should [11]. Supertest is a companion to Jest, where it is used in testing HTTP API endpoints to confirm that the backend services communicate effectively and return expected responses [11].
- **Playwright:** This tool is used for **Web UI and End-to-End (E2E) testing** by automating browser interactions, simulating real user journeys. This ensures that all front-end features, such as user login, adding products to the cart, and navigating the site, work as expected across different browsers and devices [10].

2.5.2) Configuration and Settings:

- **Playwright** is configured to run **End-to-End (E2E)** tests on the **web UI**. The tests are written as **.spec.ts files** to simulate real user actions and verify that the website functions as expected. This includes testing critical workflows like login, product display, and checkout, ensuring that users can navigate the site seamlessly. Playwright also captures **videos and screenshots** to give developers a clearer view of the web experience and help with debugging that APPENDIX Figure 11 will give a clear demonstration of how it works.
- **Jest and Supertest** are configured to test backend API calls and database interactions. The tests are written as **.js files** to verify that all backend services are functional and properly connected. This setup ensures that critical server-side functionality, like API endpoints and database communication, is thoroughly validated. The example of how Unit and Integration tests will be shown in **Figure 12 in the APPENDIX**. Additionally, for testing these Integration tests, the DevOps team utilizes an in-memory MongoDB instance provided by Mongoose to simulate database interactions, ensuring testing data will be isolated from the production database, as shown in **Figure 13 in the APPENDIX**.

By using Jest, Supertest, and Playwright, the team automates both backend and frontend testing, ensuring comprehensive coverage and reliability for Tom's web application.

2.6) Automation Tool:

To resolve Tom's **difficulty tracking deployments** and **deployment risk**, we standardized the entire CI/CD lifecycle on **Jenkins [3]**. Centralized orchestration delivers **observability**, **repeatability**, and **governed rollbacks**, reducing operational risk and improving engineering efficiency. Simplify, our team aim to 1 button process, building all infrastructure with Ansible and all process will automate on Jenkins - Tom just need to merge to "main" branch and if everything is good, the production will be hosted with new code

2.6.1) Solution Overview

- **Tools Used:** Ansible, Jenkins (Master-Slave Architecture), numerous Jenkins Plugins.
- Automated building infrastructure: Throughout setting up on Ansible yaml file, Tom don't need to manual build any in aws system or webhook github - just need to input requirement data in .env and process will automated working
- **Event-driven trigger:** GitHub **webhook** invokes Jenkins on each push.
- **Gated Pipeline (End-to-End Flow):**
 1. **Source Control** → Checkout code from GitHub
 2. **Unit Testing** → Jest framework validation
 3. **Integration Testing** → Supertest API and database validation
 4. **Container Build** → Build and tag Docker images
 5. **Registry Push** → Push images to DockerHub
 6. **Staging Deployment** → Deploy to staging environment
 7. **End-to-End Testing** → Playwright UI testing with video/screenshot capture
 8. **Promotion Gate** → Quality gate for production approval
 9. **Production Deployment** → Deploy to production environment
 10. **Notification** → Post-deployment alerts and reporting
- **Traceability:** Each run record commits SHA, build number, console logs, test **artifacts**, and evidence reports.
- **Controlled rollback:** Rollback by redeploying the last green tag or kubectl rollout undo.
- **Throughput and scalability:** Controller/agent model runs builds/tests **in parallel** on EC2 agents.

- **Operational visibility:** Email alerts and trend dashboards surface status, duration, and failure modes.

2.6.2) Configurations and Settings

- **Ansible YAML file:** Infrastructure setup defined in code format as shown in Figure 15
- **Jenkinsfile (Pipeline-as-Code):** Defines all pipeline stages, quality gates, and automated actions; stored in repository root (*see Figure 5, Appendix*)
- **Multibranch (Declarative):** Separates pull request and branch builds; protects main branch with staging promotion gate
- **GitHub Webhook + Git Plugin:** Webhook triggers Jenkins on code push and automatically checks out the exact code revision (*see Figure 6, Appendix*)
- **Agents and labels:** Linux EC2 instances configured with Node.js, Docker, and Helm for parallel processing
- **Container registry:** Automatically builds, tags, and pushes frontend/backend images to DockerHub using secure credentials
- **Kubernetes deployment:** Uses **kubectl apply** and **helm upgrade** for staging deployment; production deployment only occurs after successful **E2E testing**
- **Test Results Management:** Archive **Jest/Supertest** results; publish **Playwright** videos/screenshots and HTML reports for audit trails
- **Alert System:** Automated notifications email when builds fail or succeed
- **Audit and Retention:** Maintains history of **N** successful builds; records **approver identity** for production deployments (when manual approval is required)

2.7) Configuration Management:

This project used Ansible to **automate the setup and configuration of EC2 instances**, ensuring *consistency across servers*. Additionally, the *installation and configuration* of services such as **Docker, Kubernetes, and AWS CLI** were also automated using Ansible, allowing the DevOps team to better *manage the infrastructure* than they could with a manual approach. This reduced manual effort, generalized the project to reduce the risk of conflicts, and made it easier to scale the environment as the project grew.

2.7.1) Solution Overview:

- **Tools Used:** Ansible.

- Automates server setup and ensures configuration consistency across all environments. This makes the infrastructure stable, repeatable, and easy to manage.

2.7.2) Configuration:

- **Ansible Playbooks:** Playbooks ensure the configuration is **identical across all instances**. So these files are made to set up details for tools and services to install automatically at runtime as **quickly and efficiently as possible**. For example, the *provision-aws.yml* file provisions EC2 instances and installs Docker and Kubernetes as shown in **Figure 16 in APPENDIX**.
- **Ansible Inventory:** This specifies the **staging** and **production** environments, defining which servers should receive specific configurations. This allows **Ansible** to target the correct instances for different tasks.
- **AWS CLI Integration:** Using **AWS CLI**, **Ansible** provisions **EC2 instances** with the required **AMI IDs**, **instance types**, and **security settings**. This ensures that each EC2 instance is configured automatically as defined in the *provision-aws.yml* file. This setup eliminates manual configuration steps and ensures consistency across environments.

2.8) Automated Alerting System:

For fast incident response, devops teams' pipeline will use Jenkins Email Extension and for automatic alerts. When any stage (build/test/deploy) fails or when the build succeeds, the system will automatically send instant notifications to help Tom and the team diagnose and fix early, helping reduce downtime, cut waste, reduce stress, and risk of burnout because everyone gets alerts early.

Solution Overview:

- **Tools Used:** Jenkins Email Extension
- **Notification Rules:** Send notifications to all team members for testing; optional success notification after production deployment.

Configuration and Settings:

- **Plugins and Credentials:** Install Email plugin; store SMTP and Slack token/webhook in Jenkins Credentials as shown in **figure 17 in APPENDIX**.

By providing immediate notifications with clear issues, Tom gets immediate action signals, minimizing **Mean Time to Recovery** and deployment risk.

2.9) Orchestration

Kubernetes acts as a container orchestrator **by deploying, scaling, and managing** the application's **Docker containers**. The management of application containerization on **Kubernetes** can be further simplified by using **Helm** as a **package manager**. Helm packages all the required **Kubernetes manifests** (deployments, services, etc.) into a single, reusable package, simplifying the deployment, making it easier to *change configurations* (e.g., scale-out replication), and providing a consistent and *reliable rollback mechanism* if an update does not work or isn't desired.

2.9.1) Solution Overview:

- **Tools Used:** Kubernetes, K3s, Helm.
- Automates the deployment, scaling, and management of the containerized application, while Helm simplifies the complexity and provides robust release management capabilities like rollbacks.

2.9.2) Configurations & Settings:

- **Kubernetes Cluster:** To support the container services, we are using a K3s cluster configured on our AWS EC2 instances to allow for lightweight orchestration with better performance using minimal resources.
- **Helm Charts:** A custom Helm chart is created for the application. This Helm chart contains templated Kubernetes YAML files and a values.yaml file, as shown in **Figures 7 and 8 in the APPENDIX**, that makes it more straightforward to configure variables such as image tag, replica count, and service ports.
- **Jenkins Integration:** In the Jenkins pipeline, a command such as `helm upgrade - install` is used in the Jenkinsfile to deploy or update the application in the K3s Kubernetes cluster. This deployment and management is done seamlessly directly from Jenkins. In the end, most changes in the configuration are reflected directly in the Kubernetes application.

III. Advanced Requirement

3.1) Infrastructure as Code (IaC):

Infrastructure as Code (IaC) is an important approach that the DevOps team has used with the help of two main tools, **Ansible and AWS CLI**. Thanks to it, instead of having to execute AWS-related commands in the AWS console on the web application, the entire cloud infrastructure has been **defined and managed through code in the source code, increasing control and ease of modification later**. The first pipeline uses scripts with **AWS CLI** to

provision raw resources such as EC2 instances. Then, **Ansible playbooks** take over to configure these instances with the necessary software and settings. This approach **helps automate the entire process** of setting up the infrastructure, version control in Git, and is *easy to repeat*, which is important for disaster recovery or creating new environments.

3.1.1) Solution Overview:

- Tools Used: AWS CLI, Ansible
- The entire cloud infrastructure is managed through version-controlled code, making it automated, repeatable, and auditable. This eliminates manual setup errors and enables rapid environment creation.

3.1.2) Configurations & Settings:

- **Provisioning Scripts:** Shell scripts in the repo utilize the AWS CLI to provision EC2 instances from a stored AMI.
- **Configuration Playbooks:** Ansible playbooks, also in Git, define the complete configuration for each server type as shown in **Figure 18 in the APPENDIX**.
- **Jenkins Automation:** The pipeline orchestrates the two steps in that order, first provisioning and then configuration.

3.2) Monitoring and Alerting:

A **robust monitoring stack** built on **Prometheus and Grafana** offers **real-time insights** into the **health of the application (Figure 21 in the APPENDIX)**, which directly solves **Tom's concern regarding limited observability**. Prometheus collects **time-series metrics from all services** (CPU consumption, request latency, error rates, etc.) on an ongoing basis. Prometheus sends this data to Grafana, which **visualizes it in interactive dashboards** to provide visibility. With this data, the team can take **proactive measures rather than reactive measures** (fixing crashes after they occur). For example, seeing trends in performance to anticipate bottlenecks for catching failures that haven't happened yet. Additionally, the team can set up **warnings in Prometheus** so they are notified if they see any activity that **appears abnormal** (e.g., if a sudden spike in the error rate occurs).

3.2.1) Solution Overview:

- **Tools Used:** Prometheus, Grafana.
- Provides deep, real-time insights into application and system performance. This enables proactive problem-solving, faster debugging, and greater system stability.

3.2.2) Configurations & Settings:

- **Prometheus:** Deployed in the Kubernetes cluster and configured to automatically discover and scrape metrics from application pods. An Alertmanager is set up to handle alerts.
- **Grafana:** Connected to Prometheus as its data source. Custom dashboards are built to display key performance indicators (KPIs) for each microservice.

3.4) Advanced Deployment Strategy:

The **Blue-Green Deployment approach** is used to enable **zero-downtime releases** and **minimize deployment risk**. Two identical production environments are running concurrently, aptly named Blue (the current version serving live traffic) and Green (the new version, currently hidden from users). The pipeline deploys the new version of the application to the **Green environment**. Once testing is complete, a single, easy **network-level change** can be made to migrate all current live user traffic from Blue to Green. The old Blue environment is available in the rare event of a failure and can be **rolled back to old Blue** with a single click, ensuring a reliable rollback process. This deployment strategy helps to minimize any concerns Tom may have about risky releases and ensures recovery.

3.4.1) Solution Overview:

- **Tools Used:** Kubernetes (Services/Ingress), Helm.
- Enables zero-downtime application updates and provides instant, low-risk rollback, ensuring maximum availability and a safe release process for the project, allowing dev teams to spot issues in a near-prod environment but fix them without impacting their product.

3.4.2) Configurations & Settings:

- **Kubernetes Services:** Traffic is directed through altering Kubernetes service selectors to point traffic toward the Blue or Green deployment pods, as shown in **Figure 19 in the APPENDIX**.
- **Helm for Rollbacks:** Helm rollback <release-name> is used as a fail-safe to quickly restore the application's state in Kubernetes, as shown in **Figure 20 in the APPENDIX**.
- **Jenkins Pipeline Logic:** The pipeline includes steps for deployment in the Green environment, performing smoke tests, and then executing the script to reroute the live traffic.

3.5) Scalable Infrastructure with Auto-Scaling

The infrastructure for Tom's application is designed to be scalable using **AWS EC2 Auto Scaling** and **Kubernetes Horizontal Pod Autoscaling**. As traffic increases, **EC2 instances**

automatically scale up to meet the demand, while **Kubernetes** ensures that the appropriate number of pods are running for each service. This approach guarantees that resources are dynamically adjusted, minimizing cost while ensuring high performance.

By combining **AWS Auto Scaling** with **Kubernetes** and **Helm**, the DevOps team ensures that Tom's eCommerce platform remains highly available and scalable, even during high-demand periods. This setup allows the platform to grow without manual intervention, ensuring a seamless user experience.

IV. Application Flow

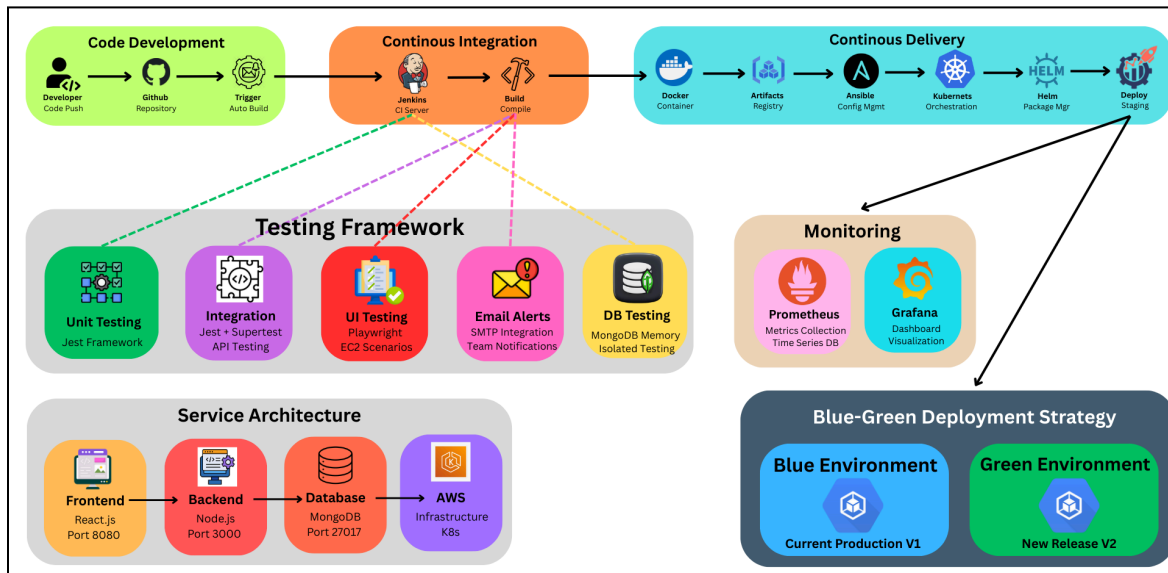


Figure 6:

4.1) Technical Workflow Architecture

4.1.1) Automated Activation Through Intelligent Integration

The pipeline architecture demonstrates advanced DevOps engineering principles. When developers commit code to the GitHub repository, the system automatically initiates a cascade of automated processes that transform source code into a fully operational e-commerce platform accessible to end users.

4.1.2) Complete Process Flow: Development to Production

1. Code Development Phase - Innovation Foundation The development workflow begins with developers implementing features and functionality. A standard **git push** operation triggers the entire automated deployment sequence without requiring manual oversight or intervention.

2. Continuous Integration - Quality Assurance Gateway The Jenkins CI server immediately activates upon code repository changes, orchestrating comprehensive build processes. The system compiles, validates, and prepares code for deployment through automated procedures that eliminate human bottlenecks and processing delays.

3. Testing Framework - Multi-Layer Quality Validation The pipeline implements a sophisticated testing approach encompassing multiple validation layers:

- **Unit Testing:** Individual component validation at the functional level
- **Integration Testing:** API endpoint stress-testing, ensuring seamless inter-service communication
- **UI Testing and E2E:** Complete user journey simulation guaranteeing optimal customer experiences
- **Database Testing:** Data integrity maintenance through comprehensive validation protocols
- **Email Alerts:** Automated team notification system enabling rapid issue resolution

4. Continuous Delivery - Deployment Excellence The containerized architecture leverages Docker, Kubernetes, and Helm technologies to create a robust deployment system. Applications undergo packaging, orchestration, and deployment through precision-engineered automated processes.

4.1.3) Blue-Green Deployment Strategy Implementation

The system maintains two identical production environments designated as Blue and Green. While customers interact seamlessly with one environment, the system deploys, tests, and optimizes updates on the alternative environment. Traffic switching occurs instantaneously when deployments complete, resulting in zero downtime and zero customer service disruption.

4.1.4) Real-Time Monitoring Infrastructure

The Prometheus and Grafana monitoring stack provides comprehensive real-time system insights. Every transaction, user interaction, and system metric undergoes continuous monitoring, analysis, and optimization. The system proactively identifies and resolves potential issues before they impact customer experience

4.2) Automation Achievement: Single-Click Customer Access

The pipeline achieves complete automation from developer code commits to customer purchase capabilities. A single development team action enables customers worldwide to immediately:

- Access new product categories and features
- Experience enhanced platform functionality
- Benefit from improved system performance
- Complete transactions with confidence and reliability

This technical implementation eliminates traditional barriers between development and deployment, creating seamless integration from innovation to customer satisfaction.

V. Known Bugs/Problems

During the deployment of the CI/CD pipeline, the DevOps team faced several challenges at different stages. Below are the main issues and how they were addressed:

5.1) Deployment Issues (Jenkins & Kubernetes Integration)

Problem: The integration of Jenkins with Kubernetes presented the first significant challenge. Jenkins could not connect to Kubernetes clusters due to the Kubernetes plugin configuration being incorrectly set. This resulted in failure to deploy and caused delays in the pipeline.

Solution: The team adjusted the configuration of the Kubernetes plugin on Jenkins to authenticate and access the cluster. The master slave structure was reviewed along with role assignment specific to individual nodes, enabling deployment to operate smoothly across the Kubernetes clusters. Once the integration was configured properly, the pipeline worked as it should with both build and deploy.

5.2) CI/CD Pipeline Reliability

Problem: The Jenkins pipeline posed reliability risks. Webhooks from GitHub could fail if Jenkins was not publicly reachable, breaking the trigger flow. Test stages such as Jest and Playwright could also be flaky, blocking deployments without retries. Secrets injected through Jenkins Configuration as Code (JCasC) and Ansible introduced a further issue that if credentials were not excluded via .gitignore, they could accidentally be committed to version control. Additionally, the blue/green logic in Jenkins was vulnerable to race conditions when multiple commits were deployed quickly, and Helm rollbacks sometimes left behind orphaned pods or incorrect service selectors.

Solution: The team mitigated these issues by ensuring Jenkins had a stable and reachable endpoint for GitHub webhooks. Retry logic and test parallelisation were introduced to reduce the impact of flaky tests. Sensitive credentials were managed through Jenkins' credentials store rather than hard-coded in Ansible templates, and .gitignore was updated to avoid leaking secrets.

Finally, pipeline concurrency controls were added to serialize blue/green cutovers, and Helm rollback procedures were refined to ensure only the intended resources were reverted.

5.3) Blue/Green Deployment Bugs

Problem: Blue/green deployments were prone to failure due to label mismatch and routing issues. The system relied on the `activeColor` label being applied consistently across Deployments and Services; any typo would prevent traffic from reaching the intended pods. Rollouts that passed smoke tests could still contain hidden runtime bugs, which only became apparent after the cutover, potentially affecting users. Helm's rollback mechanism added another risk, as it sometimes reverted unrelated configuration changes or left stale ReplicaSets running.

Solution: Deployment manifests were validated with linting and Helm schema checks to ensure `activeColor` labels remained consistent. More comprehensive smoke and health checks were added to the pipeline to catch issues earlier. To prevent rollback side effects, Helm was configured with targeted rollbacks, and post-rollback clean-up jobs were added to remove orphaned resources. This improved both reliability and confidence in the blue/green strategy.

5.4) Monitoring and Alerting Gaps

Problem: The monitoring stack had weaknesses that could undermine observability. Prometheus required applications to expose a valid `/metrics` endpoint, but incorrect port mappings or missing exporters would result in incomplete monitoring data. Alertmanager notifications also depended on SMTP relays, which were sometimes blocked by Gmail or university networks, preventing alerts from being delivered. Furthermore, Grafana dashboards and configurations were not persisted by default, meaning a simple pod restart could erase monitoring history and critical troubleshooting context.

Solution: The team ensured that both client and server services consistently exposed Prometheus metrics endpoints with correct port mappings. A backup alerting channel (such as Slack integration) was configured alongside email to avoid reliance on SMTP alone. Persistent volumes were provisioned for Grafana to ensure dashboards and historical data survived pod restarts, strengthening the reliability of the monitoring setup.

5.5) TLS and Certificate Management

Problem: The cert-manager and Let's Encrypt integration sometimes failed during frequent redeployments. Let's Encrypt enforces strict rate limits, and repeated staging and production rollouts could quickly exhaust the quota, blocking further certificate issuance. The reliance on `nip.io` domains introduced another risk: the HTTP-01 challenge required by Let's Encrypt occasionally failed if Traefik ingress rules or public IP routing were misconfigured. In

such cases, TLS certificates were not issued, and staging environments became inaccessible over HTTPS.

Solution: The team configured cert-manager to use Let's Encrypt's staging environment for testing deployments, reserving the production issuer only for finalized releases. This prevented rate limit exhaustion. Traefik ingress routes were carefully validated, and an Elastic IP was assigned to the k3s master node to stabilize DNS resolution with nip.io. These changes ensured that certificates could be issued reliably and TLS remained functional across all environments.

5.6) General Risks (Resources & Security)

Problem: Several general risks affected the stability and security of the system. Without CPU and memory requests or limits, pods were vulnerable to eviction or OOMKills on smaller EC2 instances. The cluster lacked autoscaling, meaning it could not adapt to load spikes or recover gracefully from node failures. On the security side, the standalone MongoDB instance did not use TLS or authentication, leaving data exposed. Jenkins, a common attack target, was also at risk if left publicly accessible with default credentials or overly permissive security groups.

Solution: Kubernetes manifests were updated with resource requests and limits to improve scheduling and stability. While full autoscaling was not implemented, the architecture documentation highlighted Horizontal Pod Autoscaler as a future enhancement. MongoDB was secured with authentication and limited network access, and TLS was identified as a production requirement. Jenkins access was restricted to a private security group, and strong admin credentials were enforced, reducing the exposure of the CI/CD pipeline

VI. Self-evaluation and Reflection of your CI/CD pipeline

6.1) Self-Evaluation

The **CI/CD pipeline** has been a key enabler in **automating delivery** for the **RMIT Store**. By combining **Jenkins**, **Docker**, **Helm**, and **Kubernetes**, we eliminated most manual deployment steps and created a **consistent, repeatable workflow** from **code commit** to **production rollout**. The pipeline not only handles **builds, tests, and deployments**, but also integrates **blue/green strategies** and **monitoring hooks**, which provide greater **reliability** and reduce **downtime risk**. The use of **Jenkins Configuration as Code (JCasC)** and **Ansible** further enhances **reproducibility**, ensuring that both **infrastructure** and **pipelines** can be rebuilt quickly if needed.

6.2) Industry Practices Reflection

In line with **industry best practices** discussed in **guest lectures** and **DevOps case studies**, we adopted modern **automation** and **observability tools**. **Jenkins** serves as the central orchestrator for **continuous integration**, while **Playwright** automates **end-to-end testing** in a **staging environment** before **production cutover**. **Kubernetes (via k3s)** and **Helm charts** standardize deployments across environments, enabling **scalable, container-orchestrated delivery**. **Observability** is ensured through **Prometheus**, **Grafana**, and **Alertmanager**, which mirror real-world approaches to **real-time monitoring** and **proactive alerting**. Together, these practices reflect a **professional DevOps setup** that could translate directly to industry contexts.

6.3) Effectiveness in Handling Failures & Scalability

Our pipeline incorporates mechanisms to reduce the impact of **failures** and support **scaling**. **Blue/green deployments** ensure that new versions are deployed in isolation and only receive **production traffic** after passing **health** and **smoke checks**. In case of **failure**, **Helm's rollback** function restores the **previous stable version**. On the **scalability** side, **Kubernetes** and **AWS EC2** provide the ability to **horizontally scale services**, while **Traefik ingress** and **cert-manager** ensure **secure, load-balanced access**. This combination means the platform can adapt to **growing user demand** while maintaining **availability**.

6.4) Achieving Key Aspects

The pipeline delivers on the project's **critical requirements**:

- **Automation**: From **GitHub push** to **production rollout**, **Jenkins pipelines** and **Helm charts** minimize **manual intervention**.
- **Testing**: **Jest unit tests** and **Playwright end-to-end tests** catch issues before code reaches **production**.
- **Monitoring**: **Prometheus** and **Grafana** provide **continuous metrics** and **visual dashboards**, while **Alertmanager** issues **real-time notifications** on **critical failures**.
- **Scalability**: **Kubernetes resource management** and **EC2 scaling** ensure **responsiveness** during **traffic spikes**.
- **Repeatability**: **Ansible** and **JCasC** store **configurations** in **version control**, making the entire pipeline **reproducible** across environments.

6.5) Why This Solution is the Best Fit

This pipeline is the best solution for the **RMIT Store** because it combines **automation**, **reliability**, and **scalability** in a way that is both lightweight and practical for a **university project**. **Blue/green deployments** minimize **downtime** and **risk** during **releases**, **monitoring** provides **real-time visibility** into **system health**, and **automation** across **Jenkins**, **Helm**, and

Ansible ensures faster **iteration** without sacrificing **stability**. By adopting **industry-grade tools** and **workflows**, the solution creates a strong foundation that not only meets the project's **current needs** but also prepares it for **future enhancements** and **scale**.

VII. Conclusion

Throughout this real-world project, the BugOps team learned some very important lessons about **designing and implementing a process** that truly fits our needs while staying within budget — and that works for a mid-sized project. We practiced with ***Jenkins + GitHub webhooks, Docker/Kubernetes, Ansible/Helm, Prometheus/Grafana, and Jest/Supertest/Playwright***, and we planned to scale early while respecting the limitations of the free tier. From the problems we encountered — as well as the insights from the instructors and guest speakers — we refined our approach: ***automate first, test before deployment, move to staging before production, always be ready to rollback/blue-green, and monitor with alerts***. By successfully implementing the process for Tom's project, we learned how a DevOps project should work from start to finish, what risk management is necessary for a good process, and how AWS and other tools can be combined into a smooth flow. This experience was meaningful to us, helping us build practical DevOps skills that we can consider a solid step on our journey to becoming stronger programmers.

VIII. Project Responsibilities

Team Member	Primary Responsibilities	Detailed Tasks	Workload (%)
Le Duc Huy - s4040502	AWS Infrastructure & Ansible Configuration	• AWS Infrastructure Management: Provision and configuration of EC2 instances, create and manage connections to RDS databases, create S3 buckets for static assets and artifacts, security groups, and networking configuration, set up load balancers for high availability, cost optimization, and resource monitoring. • Ansible Automation: Developing server configuration playbooks, creating automation scripts for environment setup, installing packages and managing services, managing configuration file templates, creating automated deployment workflows • Environment Coordination: Staging and production environment setup, Configuring environment variables, Installing and managing SSL certificates, Creating database migration and backup strategies	20%
Phan Manh Ha - s4019549	Infrastructure as Code (CloudFormation)	• CloudFormation Template Development: Creation of AWS resource definition templates (JSON/YAML), VPC and subnet/networking infrastructure templates, Database and compute resource templates, Security and IAM role definition templates, Parameter and output configuration • Infrastructure Reproducibility: Consistent environment across staging and production, Resource versioning and rollback, Template validation and testing process, Cost estimating and optimizing resources • Integration Support: Work with Huy on AWS resource requirements, Merge templates to Ansible configurations, and assist with infrastructure monitoring setup	20%
Ho Tuan - S4026698	Kubernetes Orchestration & Monitoring	• Kubernetes Deployment Management: Container orchestration setup and configuration, Deployment manifests creation (YAML files), Service discovery and load balancing configuration, Pod scaling and resource management policies, Rolling updates and deployment strategies, ConfigMaps and Secrets management • Monitoring & Alerting Implementation: Prometheus	20%

		metrics collection setup, Grafana dashboard creation and configuration, Application performance monitoring dashboards, Infrastructure health monitoring setup, Alert rule configuration for critical metrics, Integration with email notification system. Helm chart development and Kubernetes package management. • Advanced Deployment Features: Blue-green deployment strategy implementation, Canary deployment configuration, Health checks and readiness probes setup, Resource quotas and limits configuration. • Performance Optimization: Container resource optimization, Application performance tuning, Database connection pooling optimization, Caching strategy implementation, and monitoring	
Tran Phan Hoang Phuc - s392959	Automated Testing Framework & Email Notifications	• Implementation of Testing Framework: Configuration and setup of Jest unit testing, integration testing of API endpoints with Supertest, isolated testing with MongoDB memory server, test coverage reporting and analysis, configuration of package.json scripts for executing tests • Email Notification System: Configuration of Gmail integration in SMTP server, Configuration and setup of Jenkins email plugin, Email notification templates for team success/failure of builds, Email alerts integration in CI/CD pipeline, Resolution of issues with email delivery • Development of Tests: Unit tests for utility functions and JWT implementation, password hashing, Integration testing of authentication, products, orders, and cart APIs, Management of test data and mock implementations, and integration into CI/CD test pipeline.	20%
Nguyen Doan Trung Truc - S3974820	Team lead, Technical Documentation & UI Testing	• Technical Report Preparation: Executive summary and project overview, CI/CD pipeline description with screenshots, System architecture documentation and diagrams, Implementation methodology explanation, Team responsibilities and contribution analysis, Self-evaluation and reflection sections, References and bibliography compilation • UI/E2E Testing Framework: Playwright test suite development and maintenance, End-to-end user workflow testing scenarios, Cross-browser compatibility testing, Test automation and continuous integration, UI test failure analysis and debugging • Visual Documentation: Screenshot capture for all system components, Pipeline flow diagram creation and	20%

		updates, Video demo recording and editing preparation, User interface mockups and design documentation • Quality Assurance: Code review coordination and documentation, Testing framework validation and verification, Integration testing support with Phuc's work, Final deliverable review and formatting	
--	--	--	--

IX. References

- [1] IT Revolution, "The DevOps Handbook: Introduction Part 1 and Part 2." *IT Revolution*, Accessed: Sep. 18, 2025. [Online]. Available: http://images.itrevolution.com/documents/DevOps_Handbook_Intro_Part1_Part2.pdf.
- [2] GeeksforGeeks, "Best CI/CD Tools." *GeeksforGeeks*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.geeksforgeeks.org/devops/best-ci-cd-tools/>.
- [3] GeeksforGeeks, "What is Jenkins?" *GeeksforGeeks*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.geeksforgeeks.org/devops/what-is-jenkins/>.
- [4] L. Gordon, "Jenkins: A beginner's guide," *LewGordon*, Feb. 9, 2021. [Online]. Available: <https://www.lewgordon.com/2021/02/09/jenkins>.
- [5] Jenkins, "Architecting for Scale." *Jenkins*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.jenkins.io/doc/book/scaling/architecting-for-scale>.
- [6] GeeksforGeeks, "Introduction to Docker." *GeeksforGeeks*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.geeksforgeeks.org/devops/introduction-to-docker/>.
- [7] GeeksforGeeks, "Building with Docker using Jenkins Pipelines." *GeeksforGeeks*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.geeksforgeeks.org/devops/building-with-docker-using-jenkins-pipelines/>.
- [8] IBM, "Kubernetes Overview." *IBM*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.ibm.com/think/topics/kubernetes>.
- [9] GeeksforGeeks, "Difference Between Kubernetes and Docker Swarm." *GeeksforGeeks*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.geeksforgeeks.org/devops/difference-between-kubernetes-and-docker-swarm/>.
- [10] TestAutomationTools, "Playwright Overview." *TestAutomationTools*, Accessed: Sep. 18, 2025. [Online]. Available: <https://testautomationtools.dev/playwright-overview/>.

- [11] V. Sharma, "Scalable API Testing Framework with Jest and Supertest," *Velotio*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.velotio.com/engineering-blog/scalable-api-testing-framework-with-jest-and-supertest>.
- [12] G. Pradhan, "Connecting Jenkins to Slack: A Beginner's Guide," *Dev.to*, Accessed: Sep. 18, 2025. [Online]. Available: <https://dev.to/gideonisbuilding/connecting-jenkins-to-slack-a-beginners-guide-2c7>.
- [13] Amazon Web Services, "Amazon EC2." *AWS*, Accessed: Sep. 18, 2025. [Online]. Available: <https://aws.amazon.com/vi/ec2>.
- [14] M. Kamani et al., "Prometheus Grafana: A Metrics-focused Monitoring Stack," *ResearchGate*, 2020. [Online]. Available: https://www.researchgate.net/publication/393544298_Prometheus_Grafana_A_Metrics-focused_Monitoring_Stack.
- [15] Octopus Deploy, "Blue-Green Deployment," *Octopus Deploy*, Accessed: Sep. 18, 2025. [Online]. Available: <https://octopus.com/devops/software-deployments/blue-green-deployment>.
- [16] S. K. Shah et al., "Blue-Green and Canary Deployments in DevOps: A Comparative Study," *ResearchGate*, 2019. [Online]. Available: https://www.researchgate.net/publication/388490305_Blue-Green_and_Canary_Deployments_in_DevOps_A_Comparative_Study.
- [17] A. L. Perez, "What is Helm and Why You Should Be Using It?," *BairesDev*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.bairesdev.com/blog/what-is-helm-and-why-you-should-be-using-it/>.
- [18] CircleCI, "What is Helm?" *CircleCI*, Accessed: Sep. 18, 2025. [Online]. Available: <https://circeci.com/blog/what-is-helm/>.
- [19] Ezeelive, "Advantages and Disadvantages of Ansible," *Ezeelive*, Accessed: Sep. 18, 2025. [Online]. Available: <https://ezeelive.com/ansible-advantages-disadvantages/>.
- [20] FreeCodeCamp, "What is Ansible?" *FreeCodeCamp*, Accessed: Sep. 18, 2025. [Online]. Available: <https://www.freecodecamp.org/news/what-is-ansible/>.
- [21] Amazon Web Services, "AWS CLI." *AWS*, Accessed: Sep. 18, 2025. [Online]. Available: <https://aws.amazon.com/vi/cli/>.
- [22] S. V. Sangeet, "How to Configure Webhook in GitHub and Jenkins for Automatic Trigger with CI/CD Pipeline," *Medium*, Accessed: Sep. 18, 2025. [Online]. Available:

<https://medium.com/@sangeetv09/how-to-configure-webhook-in-github-and-jenkins-for-automatic-trigger-with-cicd-pipeline-34133e9deea>.

X. Appendices

```
loop:
- name: rmit-jenkins-sg
  desc: Jenkins
  rules:
    - proto: tcp
      from_port: 22
      to_port: 22
      cidr_ip: 0.0.0.0/0
    - proto: tcp
      from_port: 8080
      to_port: 8080
      cidr_ip: 0.0.0.0/0
- name: rmit-master-sg
  desc: k3s-master
  rules:
    - { proto: tcp, from_port: 22, to_port: 22, cidr_ip: 0.0.0.0/0 }
    - { proto: tcp, from_port: 80, to_port: 80, cidr_ip: 0.0.0.0/0 }
    - { proto: tcp, from_port: 443, to_port: 443, cidr_ip: 0.0.0.0/0 }
    # k3s apiserver from VPC
    - { proto: tcp, from_port: 6443, to_port: 6443, cidr_ip: 10.10.0.0/16 }
    # flannel VXLAN between nodes
    - { proto: udp, from_port: 8472, to_port: 8472, cidr_ip: 10.10.0.0/16 }
    # kubelet metrics/exec from control-plane
    - { proto: tcp, from_port: 10250, to_port: 10250, cidr_ip: 10.10.0.0/16 }
- name: rmit-worker-sg
  desc: k3s-workers
  rules:
    - { proto: tcp, from_port: 22, to_port: 22, cidr_ip: 0.0.0.0/0 }
    # flannel VXLAN between nodes
    - { proto: udp, from_port: 8472, to_port: 8472, cidr_ip: 10.10.0.0/16 }
    # kubelet metrics/exec from control-plane
    - { proto: tcp, from_port: 10250, to_port: 10250, cidr_ip: 10.10.0.0/16 }
- name: rmit-mongo-sg
  desc: Mongo-only-from-cluster
  rules:
    - { proto: tcp, from_port: 22, to_port: 22, cidr_ip: "{{ ssh_cidr }}" }
    - { proto: tcp, from_port: 27017, to_port: 27017, cidr_ip: 10.10.0.0/16 }
```

Figure 2: Instances ' security group settings

IMAGE


toeho/rmit-store-frontend
toeho

Pulls

Stars

Last Updated

296

0

37 minutes

IMAGE


toeho/rmit-store-backend
toeho

Pulls

Stars

Last Updated

360

0

37 minutes


toeho/rmit-store-frontend
By [toeho](#) · Updated 34 minutes ago

IMAGE

☆0

↓ 296

Overview

Tags

Sort by

Newest

Filter tags

TAG

[8c35ee95e3758d91814254427541673d9471a63f](#)

docker pull toeho/rmit-store-frontend:8c35ee95e3758d91814254427541673d9471a63f

Last pushed 34 minutes by [toeho](#)

Digest

OS/ARCH

Compressed size

[aef4977817af](#)

linux/amd64

55.98 MB


toeho/rmit-store-backend
By [toeho](#) · Updated 38 minutes ago

IMAGE

☆0

↓ 360

Overview

Tags

Sort by

Newest

Filter tags

TAG

[8c35ee95e3758d91814254427541673d9471a63f](#)

docker pull toeho/rmit-store-backend:8c35ee95e3758d91814254427541673d9471a63f

Last pushed 38 minutes by [toeho](#)

Digest

OS/ARCH

Compressed size

[4802697a22d4](#)

linux/amd64

88.88 MB

Figure 3: Docker Images for Back END and Front End components

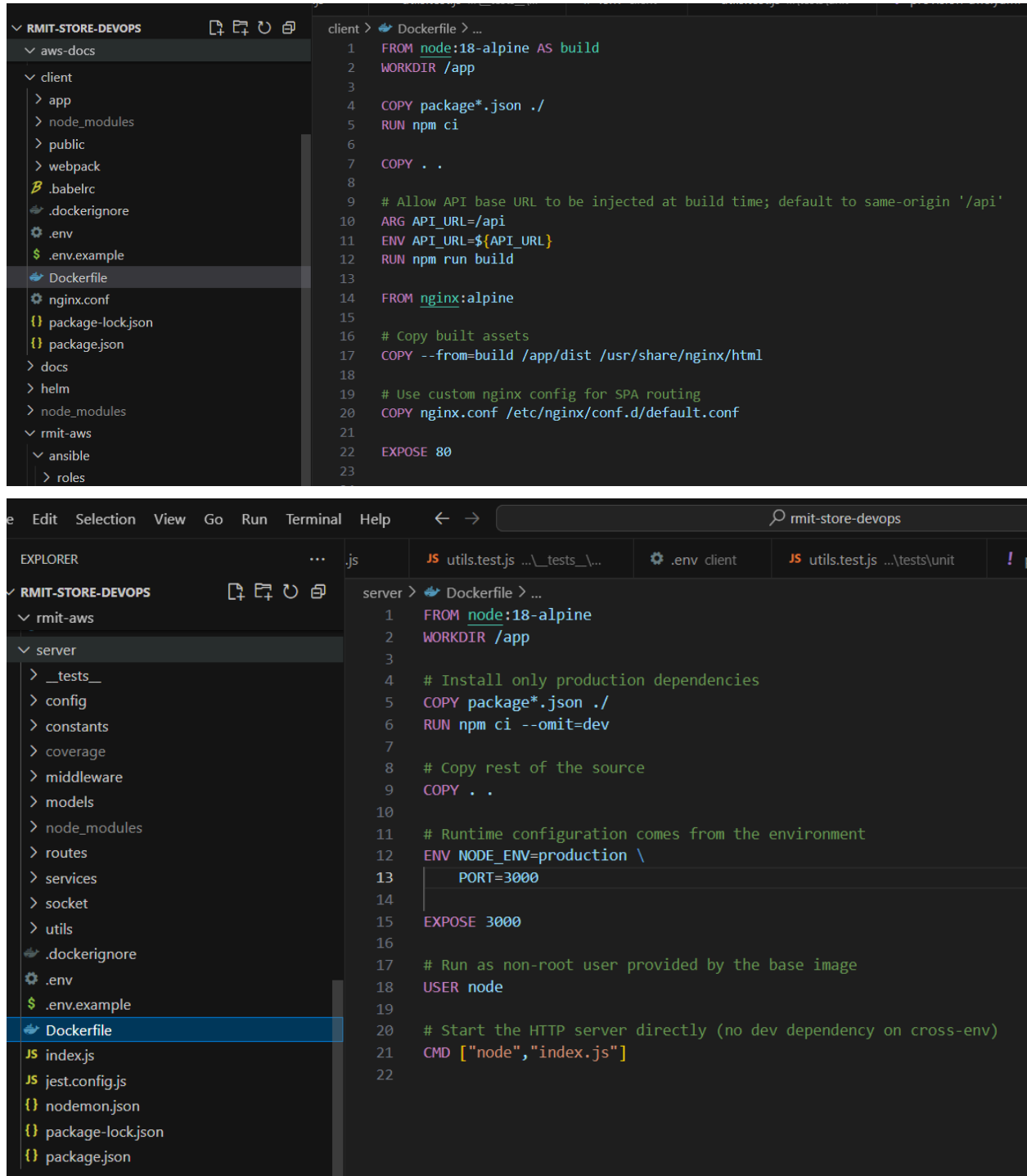


Figure 4: Client & Server Dockerfile

```
helm > rmit-store > templates > ! backend-deploy-blue.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: backend-blue
5    labels:
6      app: backend
7      activeColor: blue
8  spec:
9    replicas: {{ .Values.backend.replicas | default 1 }}
10   selector:
11     matchLabels:
12       app: backend
13       activeColor: blue
14   template:
15     metadata:
16       labels:
17         app: backend
18         activeColor: blue
19     spec:
20       {{- with .Values.backend.nodeSelector }}
21       nodeSelector:
22       {{ toYaml . | indent 8 }}
23       {{- end }}
24     containers:
25       - name: backend
26         image: {{ .Values.backend.blueImage | default .Values.backend.image | quote }}
27         imagePullPolicy: Always
28         ports:
29           - name: http
30             containerPort: {{ .Values.backend.port | default 3000 }}
31         env:
32           - name: MONGO_URI
33             valueFrom:
34               secretKeyRef:
35                 name: app-secrets
36                 key: MONGODB_URI
37           - name: JWT_SECRET
38             valueFrom:
39               secretKeyRef:
40                 name: app-secrets
41                 key: JWT_SECRET
42           - name: PORT
```

Figure 5: Helm file setup for Backend in Blue stage (1)

```

helm > rmit-store > templates > ! backend-deploy-blue.yaml
 8 spec:
14   template:
19     spec:
21       nodeSelector:
22         {{ toYaml . | indent 8 }}
23       {{- end }}
24     containers:
25       - name: backend
26         image: {{ .Values.backend.blueImage | default .Values.backend.image | quote }}
27         imagePullPolicy: Always
28         ports:
29           - name: http
30             containerPort: {{ .Values.backend.port | default 3000 }}
31         env:
32           - name: MONGO_URI
33             valueFrom:
34               secretKeyRef:
35                 name: app-secrets
36                 key: MONGODB_URI
37           - name: JWT_SECRET
38             valueFrom:
39               secretKeyRef:
40                 name: app-secrets
41                 key: JWT_SECRET
42           - name: PORT
43             value: "{{ .Values.backend.port | default 3000 }}"
44           - name: BASE_API_URL
45             value: "api"
46           - name: CLIENT_URL
47             value: "http://frontend"
48         readinessProbe:
49           httpGet:
50             path: /api/health
51             port: {{ .Values.backend.port | default 3000 }}
52             initialDelaySeconds: 5
53             periodSeconds: 5
54         livenessProbe:
55           httpGet:
56             path: /api/health
57             port: {{ .Values.backend.port | default 3000 }}
58             initialDelaySeconds: 15
59             periodSeconds: 10
60

```

Figure 6: Helm file setup for Backend in Blue stage (2)

```

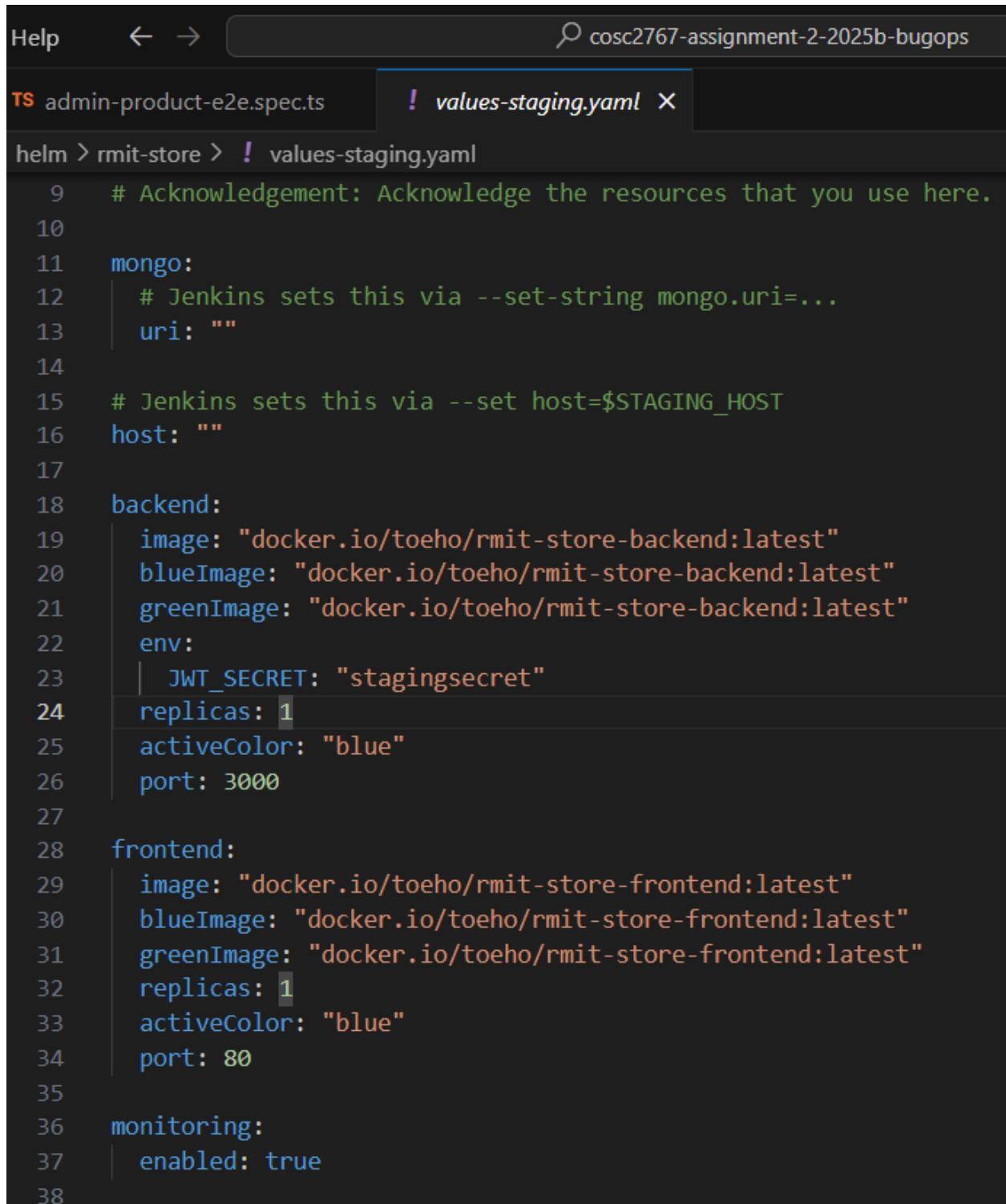
TS admin-product-e2e.spec.ts  ! frontend-deploy-blue.yaml X
helm > rmit-store > templates > ! frontend-deploy-blue.yaml
10
11 apiVersion: apps/v1
12 kind: Deployment
13 metadata:
14   name: frontend-blue
15   labels:
16     app: frontend
17     activeColor: blue
18 spec:
19   replicas: {{ .Values.frontend.replicas | default 1 }}
20   selector:
21     matchLabels:
22       app: frontend
23       activeColor: blue
24   template:
25     metadata:
26       labels:
27         app: frontend
28         activeColor: blue
29     spec:
30       {{- with .Values.frontend.nodeSelector }}
31       nodeSelector:
32       {{ toYaml . | indent 8 }}
33       {{- end }}
34     containers:
35       - name: frontend
36         image: {{ .Values.frontend.blueImage | default .Values.frontend.image | quote }}
37         imagePullPolicy: Always
38         ports:
39         - containerPort: {{ .Values.frontend.port | default 80 }}
40
41

```

Figure 7: Helm file setup for FrontEnd

```
Help  < >  cosc2767-assignment-2-2025b-bugops
TS admin-product-e2e.spec.ts  ! values-prod.yaml X
helm > rmit-store > ! values-prod.yaml
10
11 mongo:
12   # Jenkins sets this via --set-string mongo.uri=...
13   uri: ""
14
15   # Jenkins sets this via --set host=$PROD_HOST
16   host: ""
17
18 backend:
19   image: "docker.io/toeho/rmit-store-backend:latest"
20   blueImage: "docker.io/toeho/rmit-store-backend:latest"
21   greenImage: "docker.io/toeho/rmit-store-backend:latest"
22   env:
23     JWT_SECRET: "prodsecret"
24   replicas: 1
25   activeColor: "blue"
26   port: 3000
27
28 frontend:
29   image: "docker.io/toeho/rmit-store-frontend:latest"
30   blueImage: "docker.io/toeho/rmit-store-frontend:latest"
31   greenImage: "docker.io/toeho/rmit-store-frontend:latest"
32   replicas: 1
33   activeColor: "blue"
34   port: 80
35
36 monitoring:
37   enabled: true
38
```

Figure 8: Helm values-prod for running prod product



The screenshot shows a code editor with a dark theme. At the top, there's a search bar with the text "cosc2767-assignment-2-2025b-bugops". Below the search bar, there are two tabs: "admin-product-e2e.spec.ts" and "values-staging.yaml". The "values-staging.yaml" tab is active. The editor content shows a Helm values file for a chart named "rmit-store". The file includes comments and configuration for MongoDB, a backend service, a frontend service, and monitoring. Line numbers 9 through 38 are visible on the left side of the editor.

```
9  # Acknowledgement: Acknowledge the resources that you use here.
10
11  mongo:
12    # Jenkins sets this via --set-string mongo.uri=...
13    uri: ""
14
15  # Jenkins sets this via --set host=$STAGING_HOST
16  host: ""
17
18  backend:
19    image: "docker.io/toeho/rmit-store-backend:latest"
20    blueImage: "docker.io/toeho/rmit-store-backend:latest"
21    greenImage: "docker.io/toeho/rmit-store-backend:latest"
22    env:
23      | JWT_SECRET: "stagingsecret"
24    replicas: 1
25    activeColor: "blue"
26    port: 3000
27
28  frontend:
29    image: "docker.io/toeho/rmit-store-frontend:latest"
30    blueImage: "docker.io/toeho/rmit-store-frontend:latest"
31    greenImage: "docker.io/toeho/rmit-store-frontend:latest"
32    replicas: 1
33    activeColor: "blue"
34    port: 80
35
36  monitoring:
37    enabled: true
38
```

Figure 9: Helm values-staging for running staging stage

```

pipeline {
  agent any
  options {
    buildDiscarder(logRotator(daysToKeepStr: '14', numToKeepStr: '30', artifactDaysToKeepStr: '7', artifactNumToKeepStr: '10'))
    timestamps()
  }
  parameters {
    booleanParam(name: 'CLEAN_BUILD', defaultValue: false, description: 'Force docker build without cache')
  }
  environment {
    DH_NS = "${env.DH_NS ?: '<your-dockerhub-username-or-org>'}"
    FRONT_IMG = "docker.io/${DH_NS}/rmit-store-frontend"
    BACK_IMG = "docker.io/${DH_NS}/rmit-store-backend"
    STAGING_HOST = "${env.STAGING_HOST ?: ''}"
    PROD_HOST = "${env.PROD_HOST ?: ''}"
    // Speed up Docker builds if available
    DOCKER_BUILDKIT = '1'
  }
  triggers { [[ githubPush() ]] }

  stages {
    stage('checkout'){ steps { checkout scm } }

    stage('Server: Unit/API tests'){
      steps {
        dir('server'){
          sh '''
            docker run --rm \
              -v $PWD:/app -w /app \
              node:20-bullseye bash -lc "if [ -f package-lock.json ]; then npm ci; else npm install; fi && npm test -- --coverage -
            ...
          '''
        }
        // Archive coverage if present
        script { if (fileExists('server/coverage')) { archiveArtifacts artifacts: 'server/coverage/**', allowEmptyArchive: true } }
      }
    }
  }
}

```

Figure 10: Jenkins file setting up for GitHub Push with Webhook Trigger

Figure 11:

```
> e2e-playwright > TS playwright.config.ts > [e] default > use > baseURL
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30000,
  use: {
    baseURL: process.env.BASE_URL || 'http://localhost:8080',
    headless: true
  }
});
```

Figure 12: Playwright Test setup for E2E and WebUI Testing

```

ver > JS jest.config.js > <unknown> > collectCoverageFrom
1  module.exports = {
2    testEnvironment: "node",
3    verbose: true,
4    testTimeout: 30000,
5    testMatch: [
6      "**/tests/**/*.test.js",
7      "**/__tests__/intergration/**/*.[jt]s?(x)",
8      "**/__tests__/**/*.test.js",
9      "**/__tests__/**/*.spec.js",
10   ],
11   setupFilesAfterEnv: ["<rootDir>/__tests__/setup.js"],
12   collectCoverage: true,
13   coverageDirectory: "coverage",
14   coverageReporters: ["text", "lcov", "html"],
15   collectCoverageFrom: [
16     "routes/**/*.js",
17     "services/**/*.js",
18     "utils/**/*.js",
19     "models/**/*.js",
20     "**/node_modules/**",
21     "**/tests/**",
22     "**/coverage/**",
23   ],
24   testPathIgnorePatterns: ["/node_modules/", "/coverage/"],
25 };

```

```

describe("Order API Integration Tests", () => {
  let authToken;
  let userId;
  let testCartId;
  let testOrderId;

  beforeEach(async () => {
    const userResponse = await request(app).post("/api/auth/register").send({
      firstName: "Order",
      lastName: "Tester",
      email: "order.test@student.rmit.edu.vn",
      password: "Password123!",
      confirmPassword: "Password123!",
    });

    const orderUser = await User.findOne({ email: "order.test@student.rmit.edu.vn" });
    if (orderUser) {
      orderUser.role = ROLES.Admin;
      await orderUser.save();
    }
  });

```

Figure 13: File setup for Jest testing

```
r > __tests__ > JS setup.js > beforeAll() callback
// Test setup file for Jest
require('dotenv').config();

const { MongoMemoryServer } = require("mongodb-memory-server");
const mongoose = require("mongoose");
const passport = require("passport");
require('../models/user');
const jwt = require('jsonwebtoken');
let mongoServer;

require('../models/user');
require('../config/passport');
passport.initialize();

// Setup before all tests
beforeAll(async () => {
  // Create in-memory MongoDB instance
  mongoServer = await MongoMemoryServer.create();
  const mongoUri = mongoServer.getUri();
  await mongoose.connect(mongoUri, {
    useNewUrlParser: true,
    useUnifiedTopology: true,
    useCreateIndex: true,
    useFindAndModify: false,
  });

  console.log("Connected to in-memory MongoDB for testing");
});
```

Figure 14: Mongoose Clone for Intergration Testing

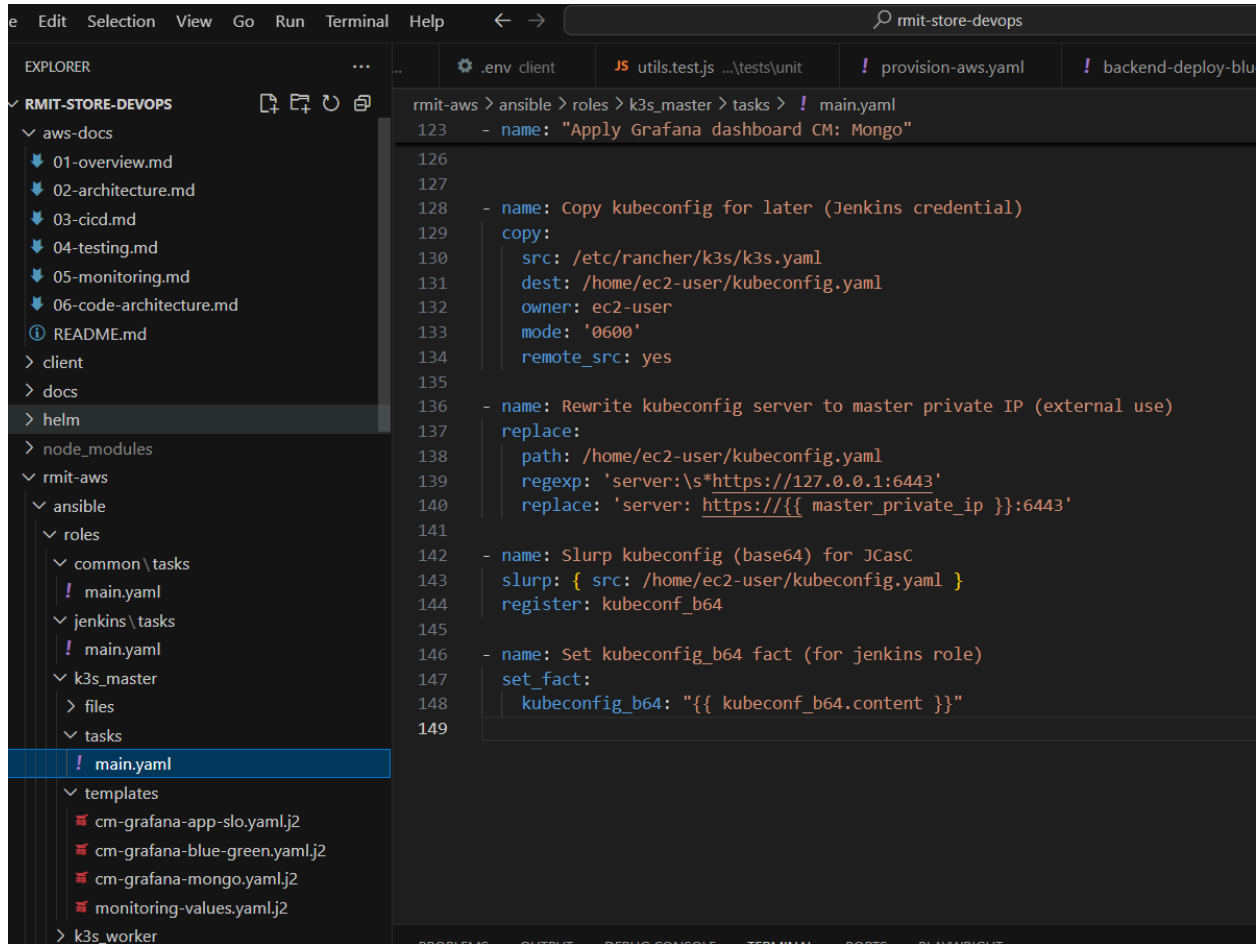
```

RMIT-STORE-DEVOPS
├── aws-docs
│   ├── 01-overview.md
│   ├── 02-architecture.md
│   ├── 03-cicd.md
│   ├── 04-testing.md
│   ├── 05-monitoring.md
│   ├── 06-code-architecture.md
│   └── README.md
├── client
├── docs
├── helm
├── node_modules
├── rmit-aws
│   ├── ansible
│   │   ├── roles
│   │   ├── ec2-outputs.env
│   │   └── ! provision-aws.yaml
│   ├── ! requirements.yml
│   ├── ! site.yaml
│   ├── > infra
│   ├── > jenkins
│   ├── $ .env.example
│   ├── M Makefile
│   ├── ① README.md
│   ├── > server
│   │   ├── > _tests_
│   │   ├── > api
│   │   └── JS health.test.js
└── > server

rmit-aws > ansible > ! provision-aws.yaml
1 - hosts: localhost
2 connection: local
3 gather_facts: no
4 vars:
5     aws_region: "{{ lookup('env','AWS_REGION') | default('us-east-1', true) }}"
6     key_name: "{{ lookup('env','EC2_KEY_NAME') }}"
7     skip_keypair: "{{ lookup('env','SKIP_KEYPAIR') | default('0', true) }}"
8     skip_ami_lookup: "{{ lookup('env','SKIP_AMI_LOOKUP') | default('0', true) }}"
9     ami_id: "{{ lookup('env','AMI_ID') | default('', true) }}"
10    type_jenkins: "{{ lookup('env','TYPE_JENKINS') | default('t3.micro', true) }}"
11    type_master: "{{ lookup('env','TYPE_MASTER') | default('t3.micro', true) }}"
12    type_worker: "{{ lookup('env','TYPE_WORKER') | default('t3.micro', true) }}"
13    type_mongo: "{{ lookup('env','TYPE_MONGO') | default('t3.micro', true) }}"
14    ami_param: "/aws/service/ami-amazon-linux-latest/al2023-ami-kernel-6.1-x86_64"
15    skip_eip: "{{ lookup('env','SKIP_EIP') | default('0', true) }}"
16    ssh_cidr_env: "{{ lookup('env','SSH_CIDR') | default('', true) }}"
17    # Root volume sizes (GiB)
18    jenkins_root_gb: "{{ lookup('env','JENKINS_ROOT_GB') | default('60', true) }}"
19    default_root_gb: "{{ lookup('env','DEFAULT_ROOT_GB') | default('16', true) }}"
20
21 collections:
22     - amazon.aws
23
24 tasks:
25     - name: Detect controller public IP (for SSH CIDR) when SSH_CIDR not provided
26       uri:
27         url: https://api.ipify.org
28         method: GET
29         return_content: yes
30         validate_certs: no
31         register: ipify
32         when: ssh_cidr_env | length == 0
33

```

Figure 15: Provision-aws.yml setting all aws infrastructure through code



```

rmit-aws > ansible > roles > k3s_master > tasks > ! main.yaml
123 - name: "Apply Grafana dashboard CM: Mongo"
126
127
128 - name: Copy kubeconfig for later (Jenkins credential)
129   copy:
130     src: /etc/rancher/k3s/k3s.yaml
131     dest: /home/ec2-user/kubeconfig.yaml
132     owner: ec2-user
133     mode: '0600'
134     remote_src: yes
135
136 - name: Rewrite kubeconfig server to master private IP (external use)
137   replace:
138     path: /home/ec2-user/kubeconfig.yaml
139     regexp: 'server:\s*https://127.0.0.1:6443'
140     replace: 'server: https://{{ master_private_ip }}:6443'
141
142 - name: Slurp kubeconfig (base64) for JCasC
143   slurp: { src: /home/ec2-user/kubeconfig.yaml }
144   register: kubeconf_b64
145
146 - name: Set kubeconfig_b64 fact (for jenkins role)
147   set fact:
148     kubeconfig_b64: "{{ kubeconf_b64.content }}"
149

```

Figure 16: Ansible Setting up file



```

post {
  success {
    echo "Deployment succeeded: ${GIT_COMMIT}"
    mail to: "${env.EMAIL_TO ?: ''}",
        subject: "Deployment Succeeded: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
        body: "Build URL: ${env.BUILD_URL}\nCommit: ${env.GIT_COMMIT}\nStaging: http://${env.STAGING_HOST}\nProd: http://${env.P
    script {
  }
  failure {
    echo "Deployment failed at stage: ${env.STAGE_NAME}"
    // Send email via Jenkins core mailer (requires SMTP configured in Manage Jenkins)
    mail to: "${env.EMAIL_TO ?: ''}",
        subject: "Deployment Failed: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
        body: "Build URL: ${env.BUILD_URL}\nStage: ${env.STAGE_NAME}"
  }
}

```

Figure 17: Mail Notification Setting

```
Resources:
  Jenkins:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !If [ UseSSM, !FindInMap [RegionToAmi, !Ref 'AWS::Region', AMI], !Ref UbuntuAmiParam ]
      InstanceType: !Ref TypeJenkins
      KeyName: !Ref KeyName
      NetworkInterfaces:
        - AssociatePublicIpAddress: true
          DeviceIndex: 0
          SubnetId: !Ref SubnetIdA
          GroupSet: [!Ref JenkinsSG]
      Tags: [{Key: Name, Value: jenkins}]

  Master:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !If [ UseSSM, !FindInMap [RegionToAmi, !Ref 'AWS::Region', AMI], !Ref UbuntuAmiParam ]
      InstanceType: !Ref TypeMaster
      KeyName: !Ref KeyName
      NetworkInterfaces:
        - AssociatePublicIpAddress: true
          DeviceIndex: 0
          SubnetId: !Ref SubnetIdA
          GroupSet: [!Ref MasterSG]
      Tags: [{Key: Name, Value: k3s-master}]

  Clientworker:
    Type: AWS::EC2::Instance
    Properties:
      ImageId: !If [ UseSSM, !FindInMap [RegionToAmi, !Ref 'AWS::Region', AMI], !Ref UbuntuAmiParam ]
      InstanceType: !Ref TypeWorker
```

```
JenkinsSG:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: Jenkins
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - { IpProtocol: tcp, FromPort: 22, ToPort: 22, CidrIp: 0.0.0.0/0 }
      - { IpProtocol: tcp, FromPort: 8080, ToPort: 8080, CidrIp: 0.0.0.0/0 }

  MasterSG:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: k3s-master
      VpcId: !Ref VPC
      SecurityGroupIngress:
        - { IpProtocol: tcp, FromPort: 22, ToPort: 22, CidrIp: 0.0.0.0/0 }
        - { IpProtocol: tcp, FromPort: 80, ToPort: 80, CidrIp: 0.0.0.0/0 }
        - { IpProtocol: tcp, FromPort: 443, ToPort: 443, CidrIp: 0.0.0.0/0 }
        - { IpProtocol: tcp, FromPort: 6443, ToPort: 6443, SourceSecurityGroupId: !Ref JenkinsSG }
        - { IpProtocol: tcp, FromPort: 6443, ToPort: 6443, SourceSecurityGroupId: !Ref MasterSG }
        - { IpProtocol: tcp, FromPort: 6443, ToPort: 6443, SourceSecurityGroupId: !Ref WorkerSG }

  WorkerSG:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: k3s-workers
```

Figure 18: AWS sSETTING UPO

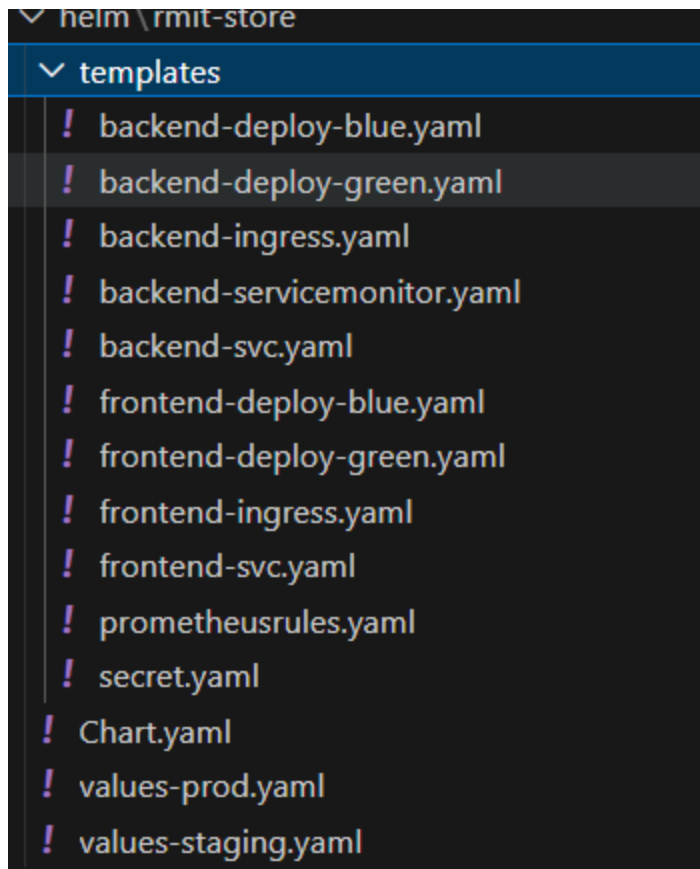


Figure 19: Blue - Green teamplate

```

stage('Smoke GREEN'){
  when { expression { return (env.E2E_OK ?: '') == '1' } }
  steps {
    sh '''
      if [ -z "$PROD_HOST" ]; then
        echo "PROD_HOST is not set. Provide via JCasC env or .env → Makefile inventory."
        exit 1
      fi
      PROD_URL="http://$PROD_HOST/api/health"
      echo "Prod smoke URL: $PROD_URL"
      docker run --rm curlimages/curl:8.8.0 -fsS "$PROD_URL"
    '''
  }
}

stage('Cutover BLUE -> GREEN'){
  when { expression { return (env.E2E_OK ?: '') == '1' } }
  steps {
    withCredentials([file(credentialsId: 'kubeconfig-master', variable: 'KUBECONF')]){
      sh '''
        set -e
        cp "$KUBECONF" kubeconfig && chmod 600 kubeconfig
        docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubect1:v1.30.6 -n prod patch service backend -p '{"spec":{"selector":{"app":'
        docker run --rm -u 0 -e KUBECONFIG=/kubeconfig -v "$PWD"/kubeconfig:/kubeconfig:ro \
          rancher/kubect1:v1.30.6 -n prod patch service frontend -p '{"spec":{"selector":{"app":'
        shred -u kubeconfig || rm -f kubeconfig
      '''
    }
  }
}

```

Figure 20: Blue-Green Stage

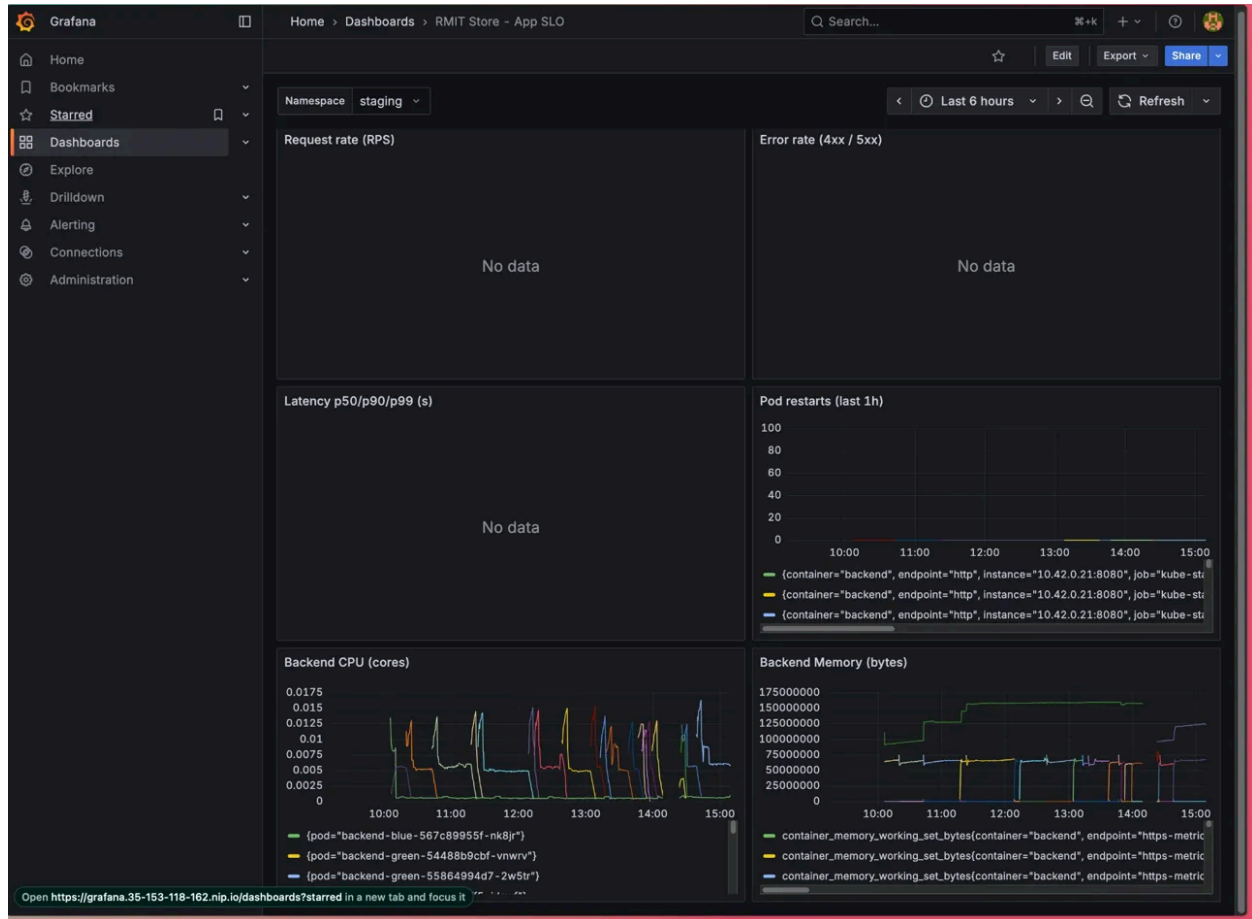


Figure 21: Grafana Dashboard